Redes Convolucionais I

Machine Learning 2 2026.1

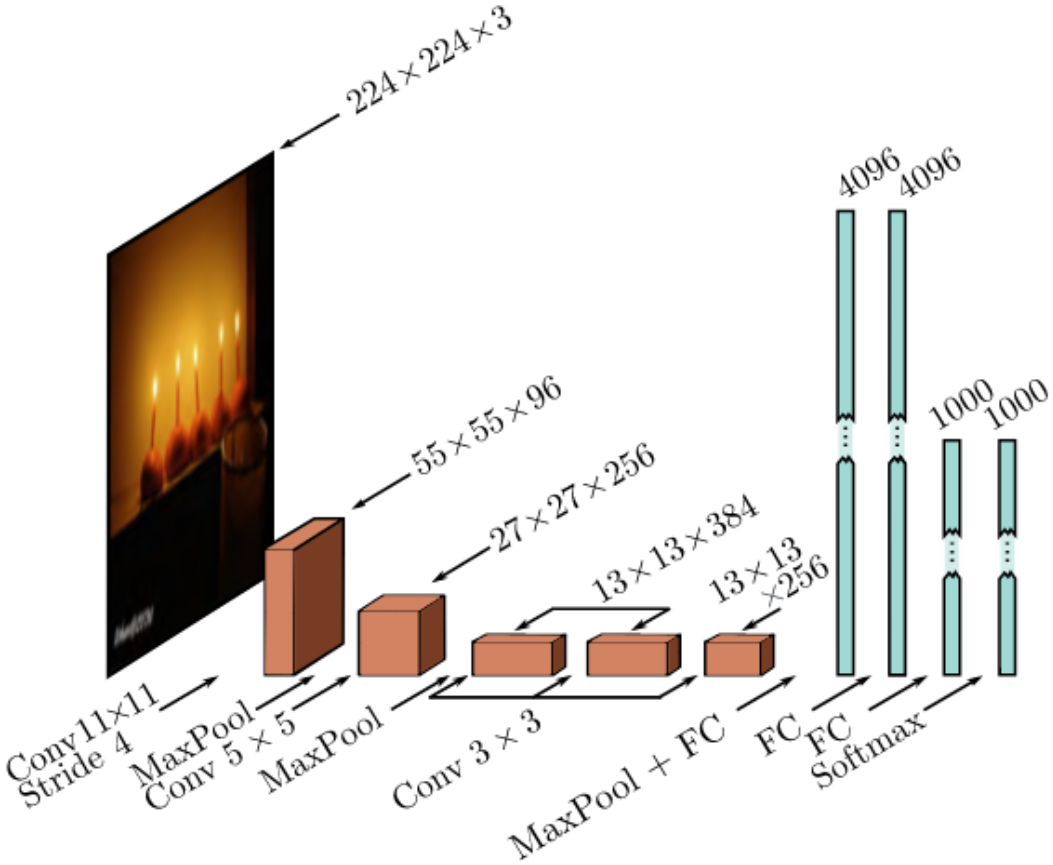
Francisco Ganacim, Daniel Yukimura

IMPA

April 1, 2026

Objetivo da Aula

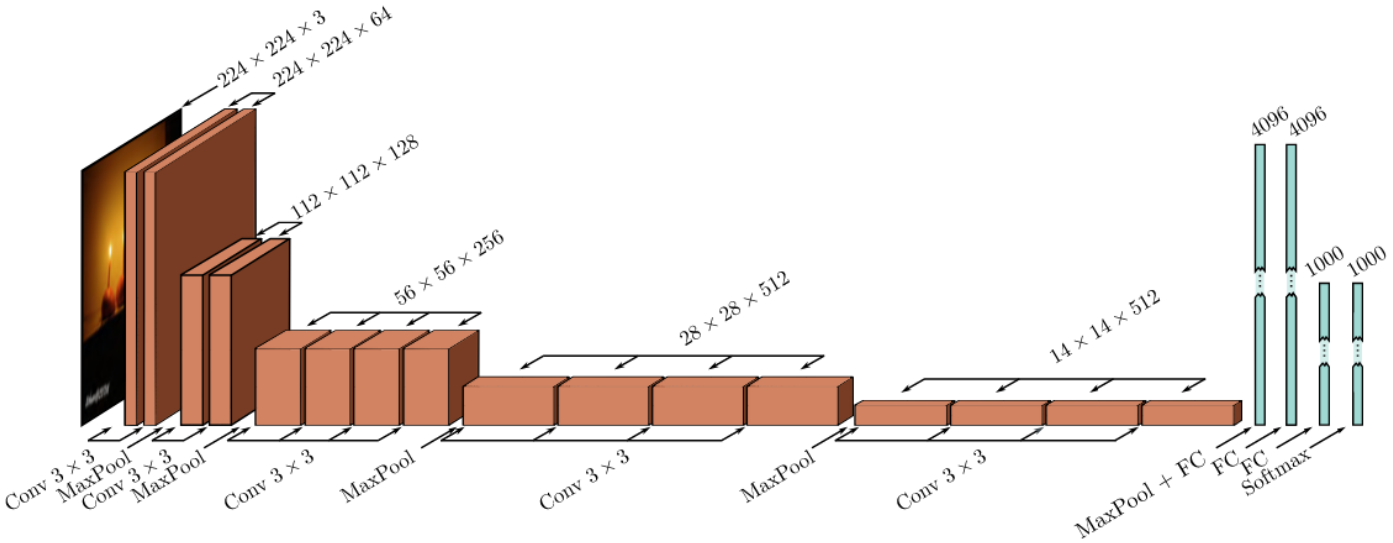
- Classificação de imagens com redes convolucionais.



[7] 2012 — Alex Krizhevsky et al. — *ImageNet Classification with Deep Convolutional Neural Networks*
[13] 2015 — Karen Simonyan et al. — *Very Deep Convolutional Networks for Large-Scale Image Recognition*
[10] 2023 — Simon Prince — *Understanding Deep Learning*

Objetivo da Aula

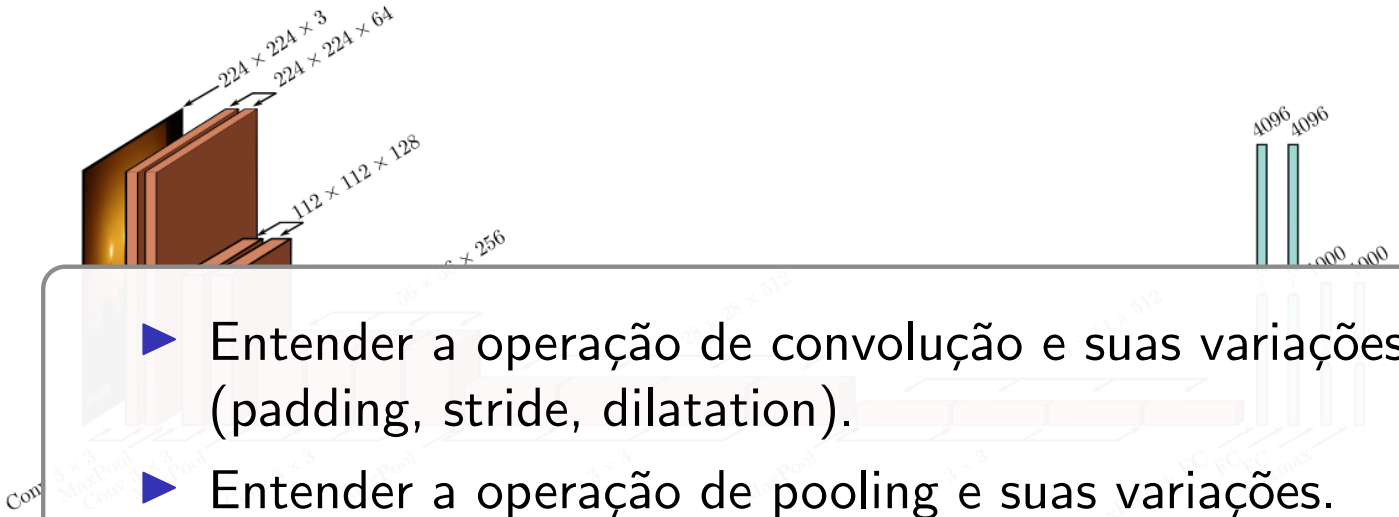
- Classificação de imagens com redes convolucionais.



[7] 2012 — Alex Krizhevsky et al. — *ImageNet Classification with Deep Convolutional Neural Networks*
[13] 2015 — Karen Simonyan et al. — *Very Deep Convolutional Networks for Large-Scale Image Recognition*
[10] 2023 — Simon Prince — *Understanding Deep Learning*

Objetivo da Aula

- Classificação de imagens com redes convolucionais.



- Entender a operação de convolução e suas variações (padding, stride, dilatation).
- Entender a operação de pooling e suas variações.
- Perdas e métricas para classificação de imagens.

[7] 2012 — Alex Krizhevsky et al. — *ImageNet Classification with Deep Convolutional Neural Networks*
[13] 2015 — Karen Simonyan et al. — *Very Deep Convolutional Networks for Large-Scale Image Recognition*
[10] 2023 — Simon Prince — *Understanding Deep Learning*

Machine Learning: Definição do Problema (Revisão)

- Aprender uma função $f : \mathcal{X} \mapsto \mathcal{Y}$ a partir de um conjunto de amostras $\mathcal{D} = \{(x_i, y_i); x_i \in \mathcal{X}, y_i \in \mathcal{Y}\}_{i=1,\cdots,N}$.
- $y = f(x, \theta)$ é um modelo que depende dos parâmetros θ.
- Queremos encontrar $\hat{\theta}$ que minimiza uma função de custo C.

$$\hat{\theta} = \arg\min_{\theta} C(\theta)$$

- Podemos definir C como:

$$C(\theta) = \frac{1}{N} \sum_{i=1}^{N} \ell(f(x_i, \theta), y_i)$$

- Onde ℓ é uma função de perda que mede a diferença entre a predição $f(x_i, \theta)$ e o valor real y_i.

Convolução: Definição

Em processamente de sinais, a convolução aparece ao estudarmos filtros. Dado um filtro $\mathcal{L} : \mathcal{X} \mapsto \mathcal{X}$, que satifaz, para os sinais I e J:

- Linearidade: $\mathcal{L}(aI + bJ) = a\mathcal{L}(I) + b\mathcal{L}(J)$
- Invariância por translação: $\mathcal{L}(I(x - t)) = \mathcal{L}(I)(x - t)$

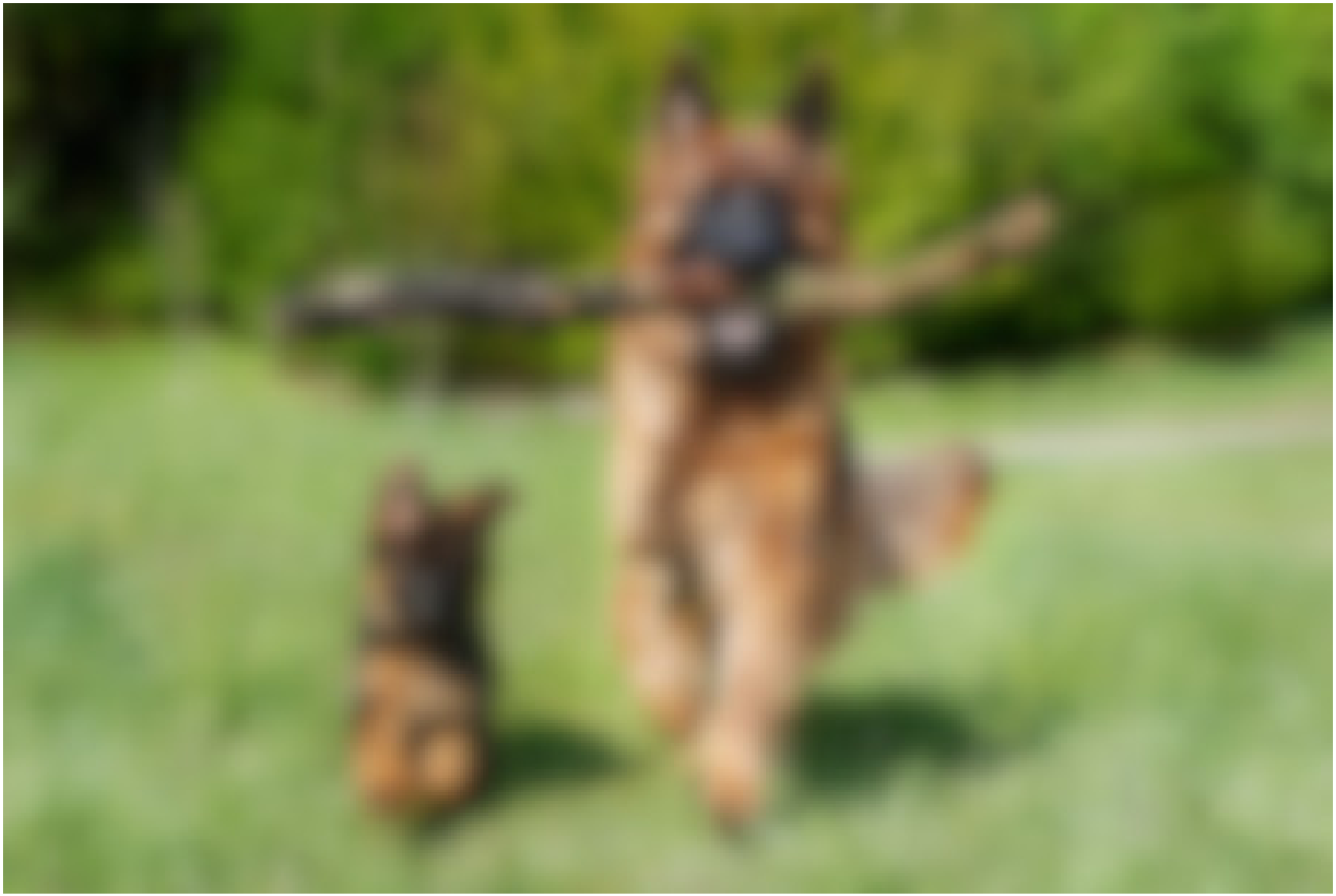
É possivel provar que

$$\mathcal{L}(I)(x) = I(x) * K(x) = \int_{-\infty}^{\infty} I(t)K(x - t)dt$$

Onde K é a resposta ao impulso do filtro $\mathcal{L}$.

Convolução como filtro de sinais

Convolução como filtro de sinais



Convolução como filtro de sinais

Convolução como filtro de sinais

Operação de Correlação

- A operação de correlação é uma operação semelhante à convolução, mas não envolve a inversão do kernel.
- Em 1D:

$$(I \star K)(x) = \int_{-\infty}^{\infty} I(t)K(x+t)dt$$

- Em 2D:

$$(I \star K)(x, y) = \int_{-\infty}^{\infty} I(t, u)K(x+t, y+u)dt$$

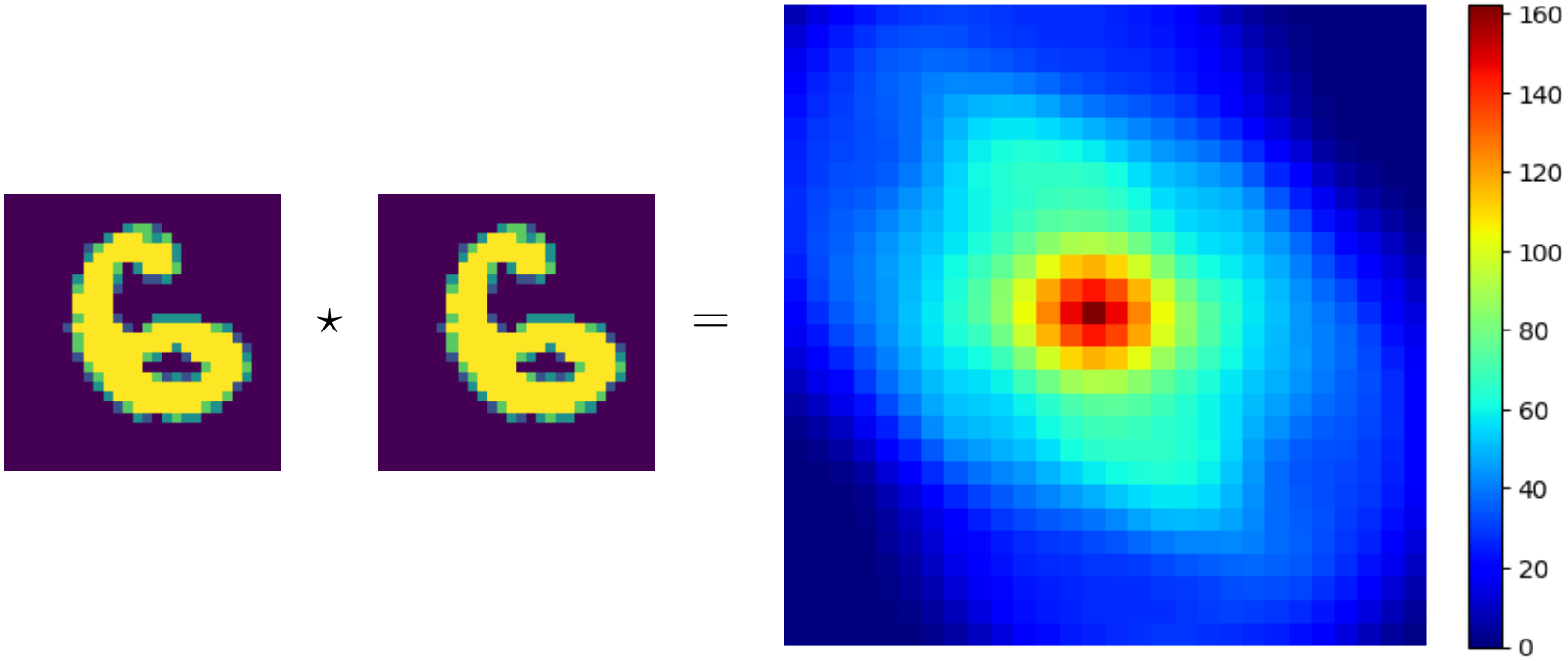
- A operação de correlação é frequentemente usada em processamento de imagem e visão computacional para detectar padrões e características em imagens.
- Quando o kernel é simétrico em relação à origem, a convolução e a correlação são equivalentes.

Correlação: Exemplo

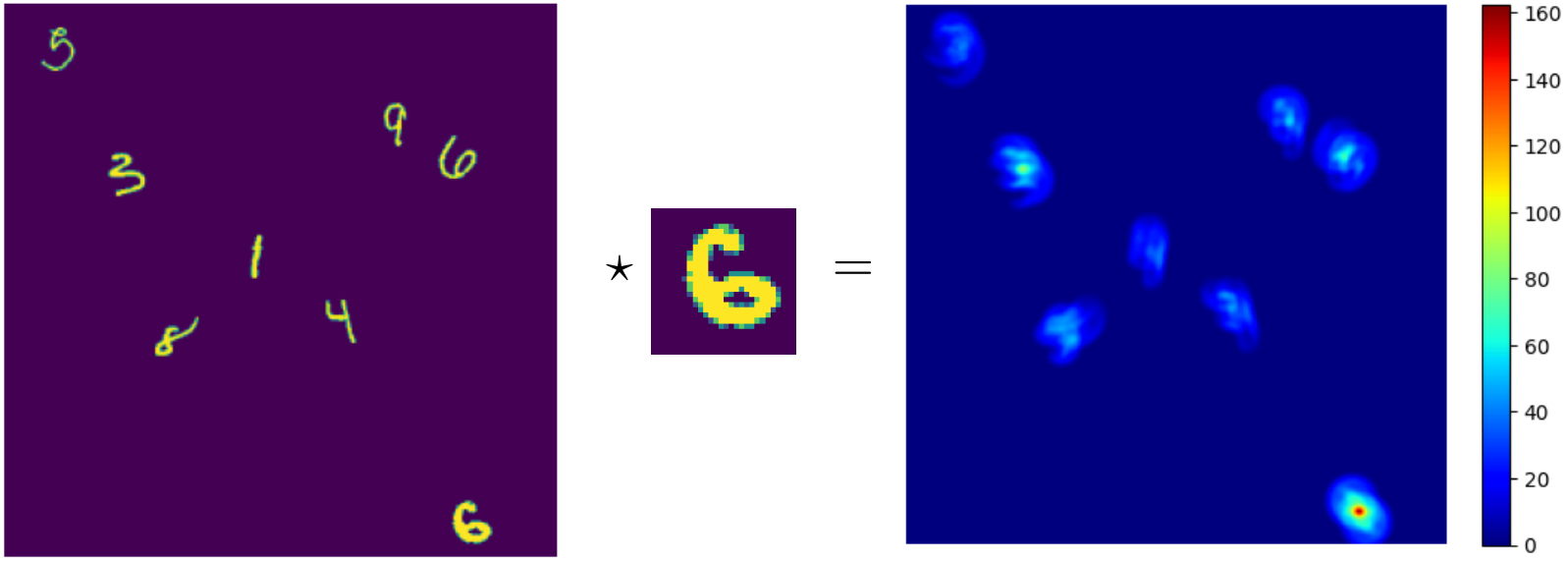
$\star$

$=$

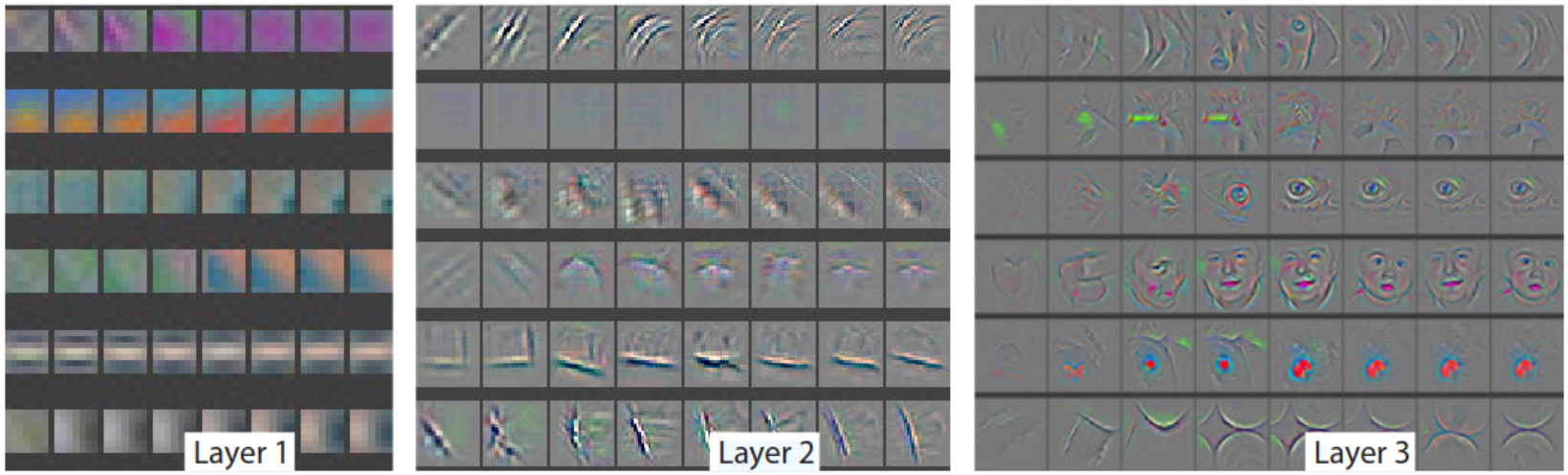
Correlação: Exemplo



Correlação: Exemplo



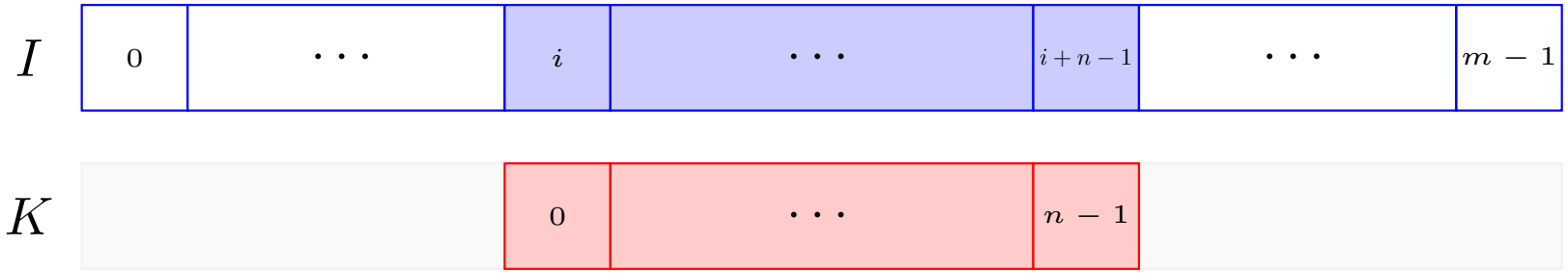
Features aprendidos por CNNs



[17] 2013 — Matthew D. Zeiler et al. — *Visualizing and Understanding Convolutional Networks*
[12] 2014 — Karen Simonyan et al. — *Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps*

Definições

- $I = [I_0, I_1, I_2, \cdots, I_{m-1}]$
- $K = [K_0, K_1, \cdots, K_{n-1}]$
- $[I \star K]_i = \sum_{j=0}^{m-1} I_j K_{j-i}$, onde $K_j = 0$ se $j \notin [0, n-1]$. Ou:
- $[I \star K]_i = \sum_{j=0}^{n-1} I_{i+j} K_j$



Exemplo 1D

-2	-5	3	0	4	0	0	-4	2	-1

-2	1	3

8							

Exemplo 1D

-2	-5	3	0	4	0	0	-4	2	-1

-2	1	3

8	13						

Exemplo 1D

-2	-5	3	0	4	0	0	-4	2	-1

-2	1	3

8	13	6					

Exemplo 1D

-2	-5	3	0	4	0	0	-4	2	-1

-2	1	3

8	13	6	4				

Exemplo 1D

-2	-5	3	0	4	0	0	-4	2	-1

-2	1	3

8	13	6	4	-8			

Exemplo 1D

-2	-5	3	0	4	0	0	-4	2	-1

-2	1	3

8	13	6	4	-8	-12		

Exemplo 1D

-2	-5	3	0	4	0	0	-4	2	-1

-2	1	3

8	13	6	4	-8	-12	2

Exemplo 1D

-2	-5	3	0	4	0	0	-4	2	-1

-2	1	3

8	13	6	4	-8	-12	2	7

Forma Matricial: Exemplo

- $I = [I_0, \cdots, I_6]$
- $K = [K_0, K_1, K_2]$

$$I \star K = \underbrace{\begin{bmatrix} K_0 & K_1 & K_2 & 0 & 0 & 0 & 0 \\ 0 & K_0 & K_1 & K_2 & 0 & 0 & 0 \\ 0 & 0 & K_0 & K_1 & K_2 & 0 & 0 \\ 0 & 0 & 0 & K_0 & K_1 & K_2 & 0 \\ 0 & 0 & 0 & 0 & K_0 & K_1 & K_2 \end{bmatrix}}_{\text{m} = 7} \begin{bmatrix} I_0 \\ I_1 \\ I_2 \\ I_3 \\ I_4 \\ I_5 \\ I_6 \end{bmatrix}$$

$\text{m-n+1} = 5$

Forma Matricial: Exemplo

- $I = [I_0, \cdots, I_6]$
- $K = [K_0, K_1, K_2]$

$$I \star K = \underbrace{\begin{bmatrix} K_0 & K_1 & K_2 & 0 & 0 & 0 & 0 \\ 0 & K_0 & K_1 & K_2 & 0 & 0 & 0 \\ 0 & 0 & K_0 & K_1 & K_2 & 0 & 0 \\ 0 & 0 & 0 & K_0 & K_1 & K_2 & 0 \\ 0 & 0 & 0 & 0 & K_0 & K_1 & K_2 \end{bmatrix}}_{\text{m} = 7} \begin{bmatrix} I_0 \\ I_1 \\ I_2 \\ I_3 \\ I_4 \\ I_5 \\ I_6 \end{bmatrix}$$

$\text{m-n+1} = 5$

1. Convolução é uma operação linear.

Forma Matricial: Exemplo

- $I = [I_0, \cdots, I_6]$
- $K = [K_0, K_1, K_2]$

$$I \star K = \underbrace{\begin{bmatrix} K_0 & K_1 & K_2 & 0 & 0 & 0 & 0 \\ 0 & K_0 & K_1 & K_2 & 0 & 0 & 0 \\ 0 & 0 & K_0 & K_1 & K_2 & 0 & 0 \\ 0 & 0 & 0 & K_0 & K_1 & K_2 & 0 \\ 0 & 0 & 0 & 0 & K_0 & K_1 & K_2 \end{bmatrix}}_{\text{m} = 7} \begin{bmatrix} I_0 \\ I_1 \\ I_2 \\ I_3 \\ I_4 \\ I_5 \\ I_6 \end{bmatrix}$$

$\text{m-n+1} = 5$

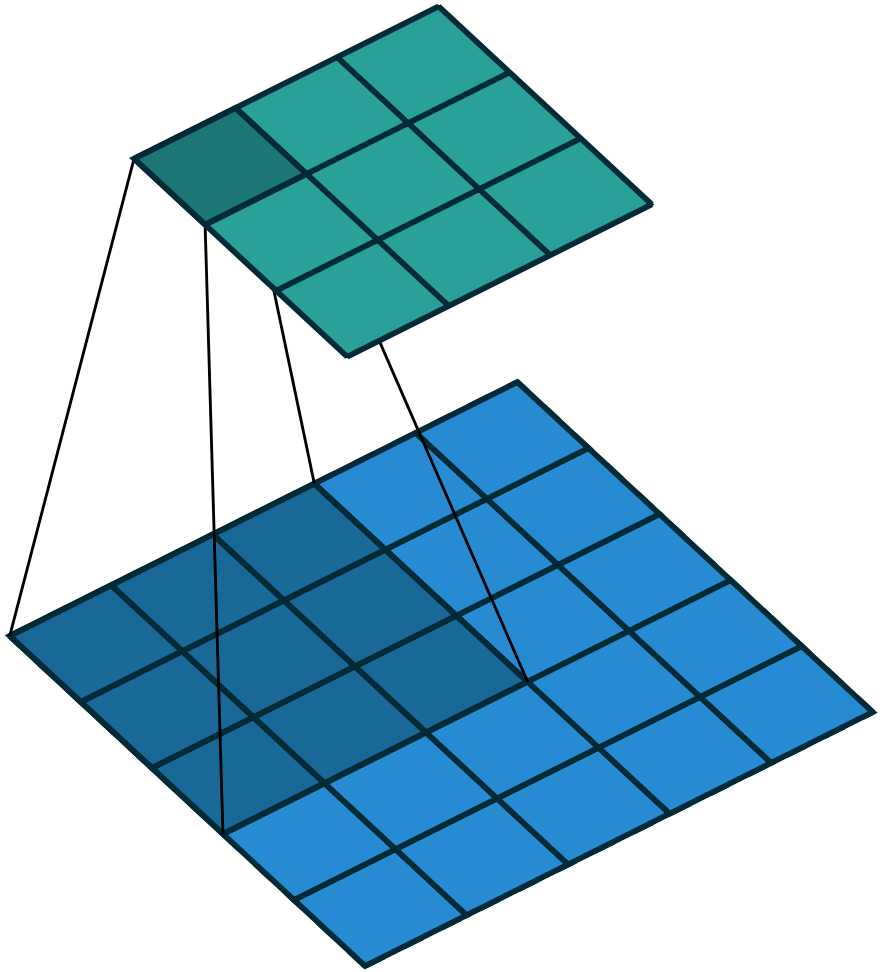
1. Convolução é uma operação linear.
2. O tamanho da saída é $m - n + 1$.

Convolução 2D

- $I = [I_{i,j}]$, onde $i \in [0, H-1]$ e $j \in [0, W-1]$
- $K = [K_{u,v}]$, onde $u \in [0, m-1]$ e $v \in [0, n-1]$

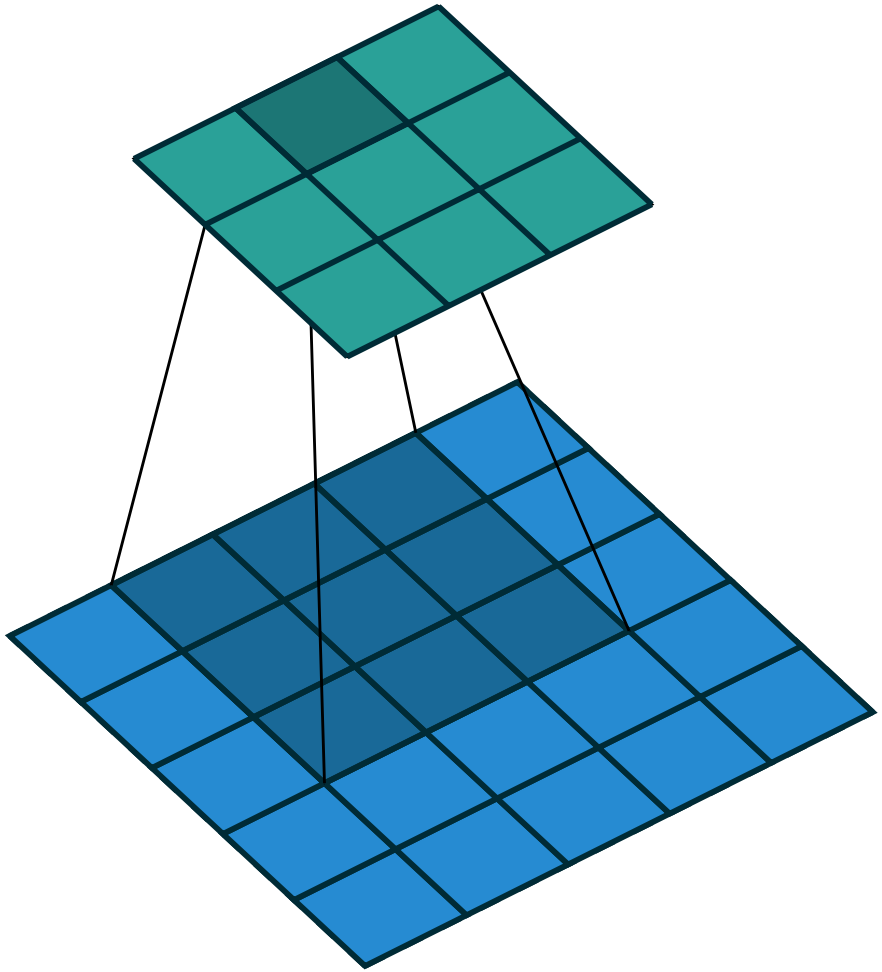
$$
\begin{aligned}
[I \star K]_{i,j} &= \sum_{i=0}^{H-1} \sum_{j=0}^{W-1} I_{i,j} K_{i-u,j-v}, \text{ ou} \\
&= \sum_{u=0}^{m-1} \sum_{v=0}^{n-1} I_{i+u,j+v} K_{u,v}
\end{aligned}
$$

Convolução 2D: Exemplo



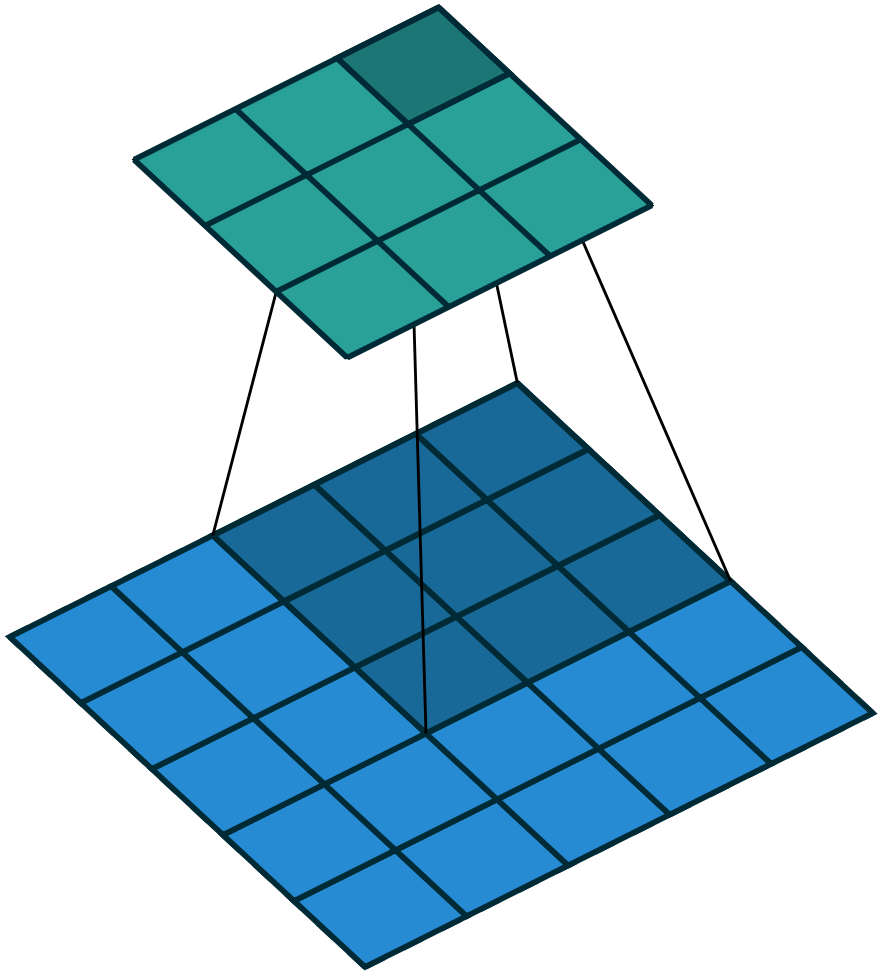
[15] 2026 — vdumoulin — *vdumoulin/conv_arithmetic*

Convolução 2D: Exemplo



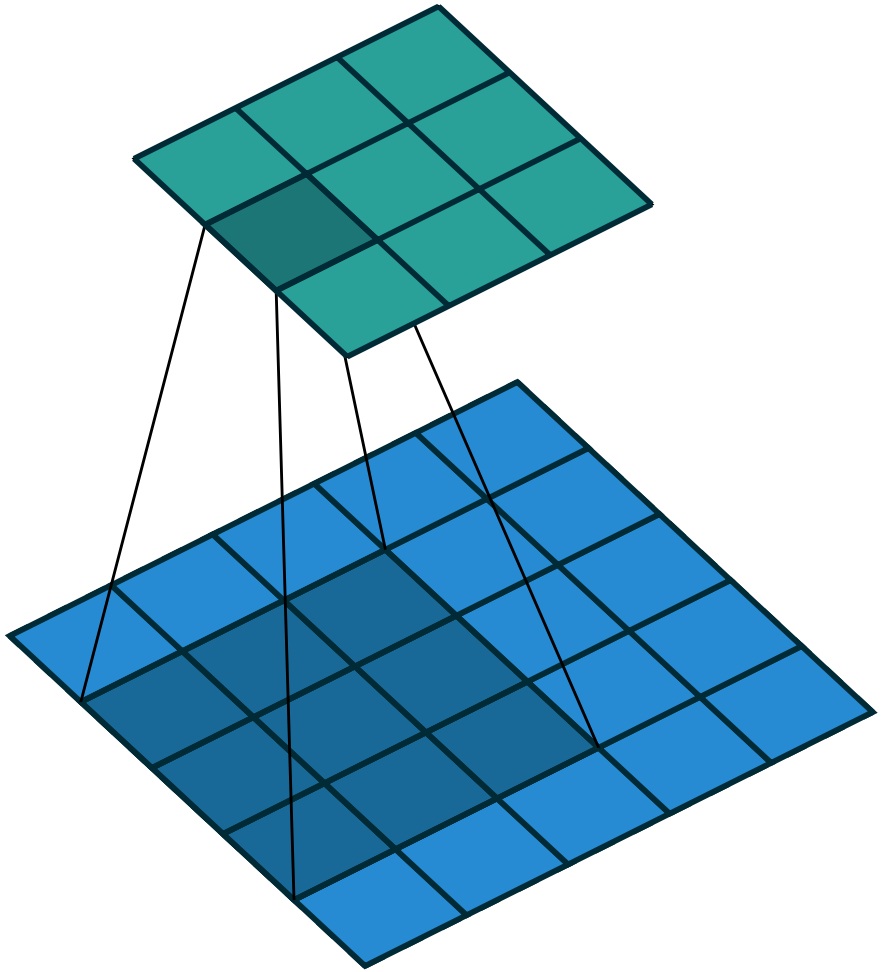
[15] 2026 — vdumoulin — *vdumoulin/conv_arithmetic*

Convolução 2D: Exemplo



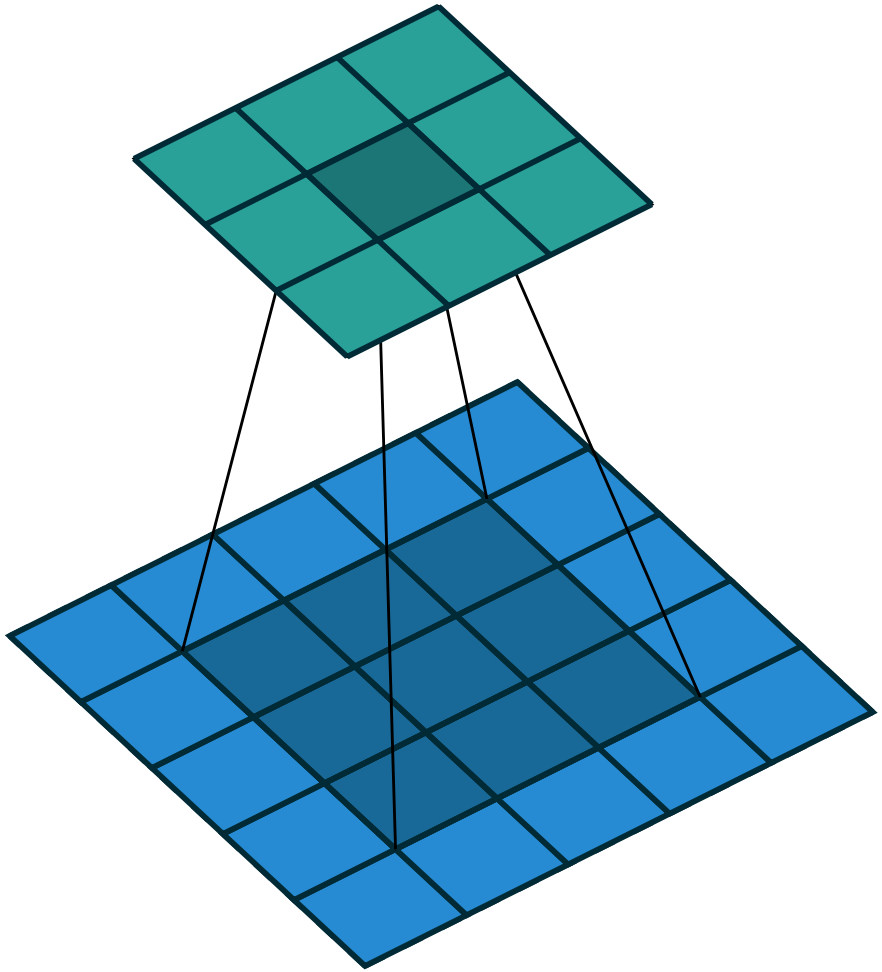
[15] 2026 — vdumoulin — *vdumoulin/conv_arithmetic*

Convolução 2D: Exemplo



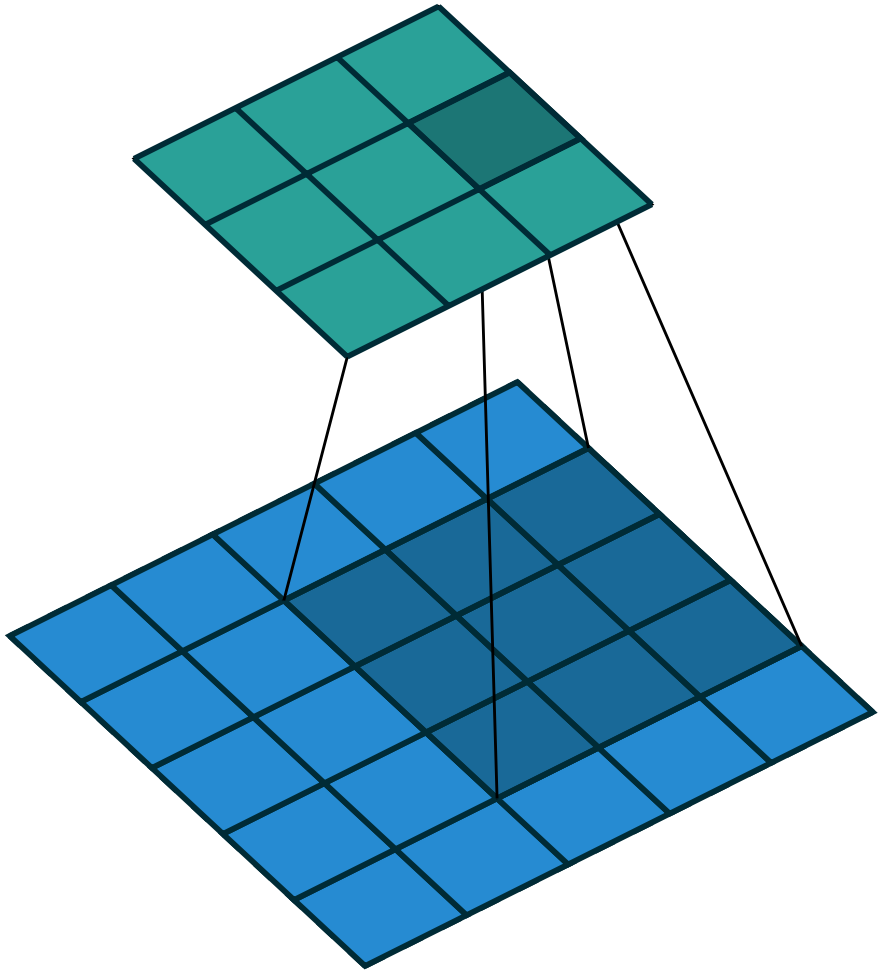
[15] 2026 — vdumoulin — *vdumoulin/conv_arithmetic*

Convolução 2D: Exemplo



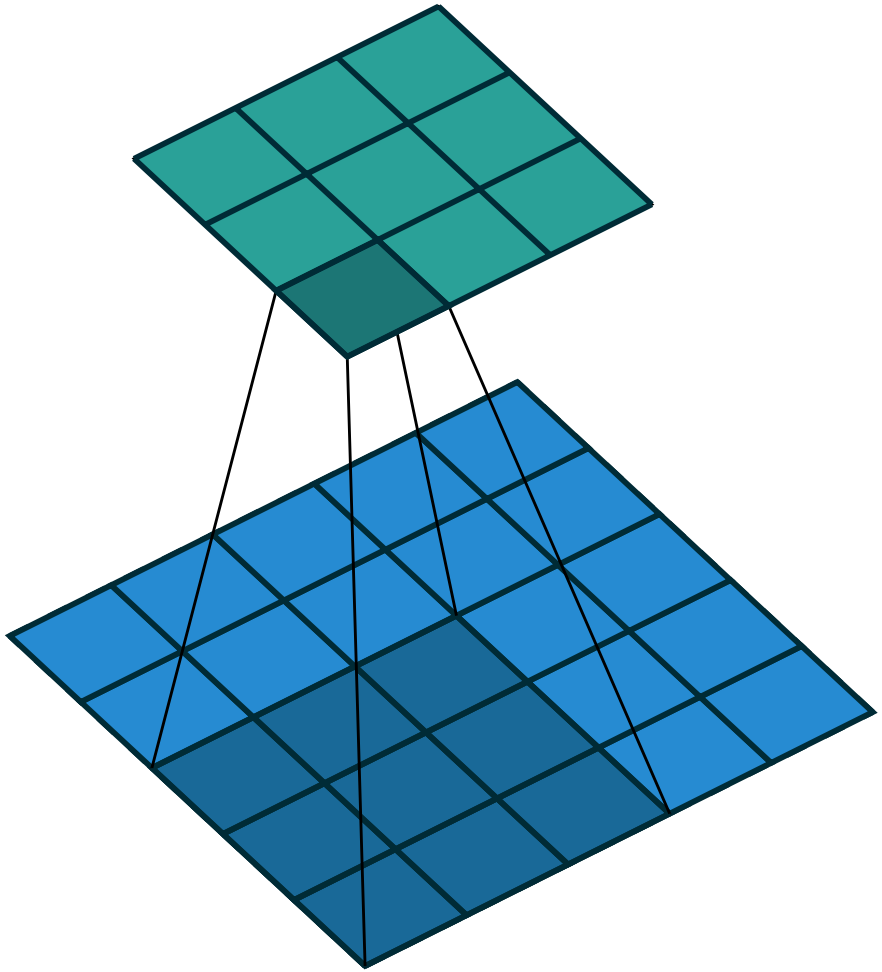
[15] 2026 — vdumoulin — *vdumoulin/conv_arithmetic*

Convolução 2D: Exemplo



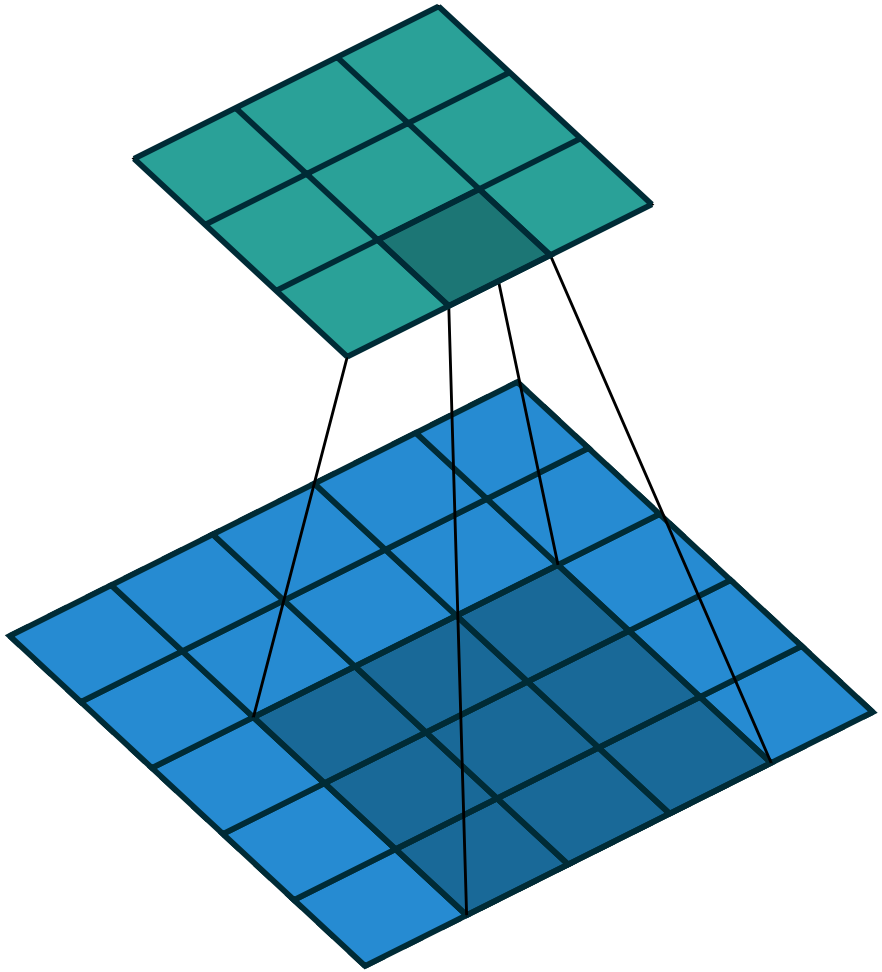
[15] 2026 — vdumoulin — *vdumoulin/conv_arithmetic*

Convolução 2D: Exemplo



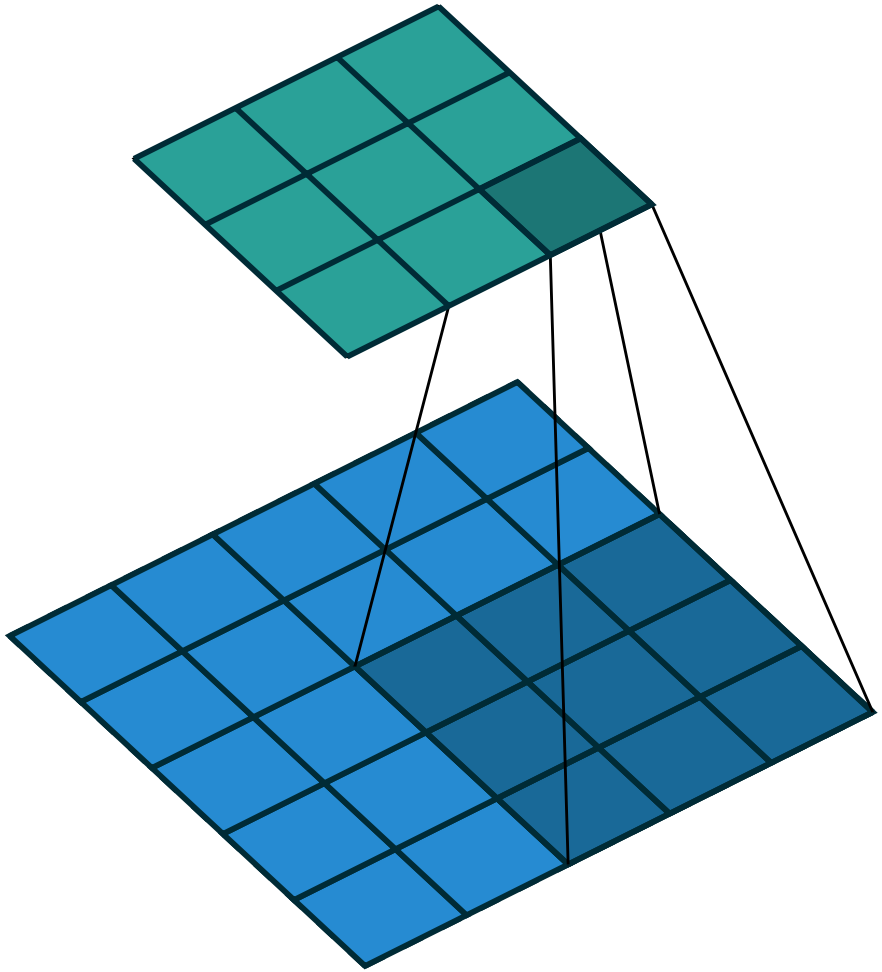
[15] 2026 — vdumoulin — *vdumoulin/conv_arithmetic*

Convolução 2D: Exemplo



[15] 2026 — vdumoulin — *vdumoulin/conv_arithmetic*

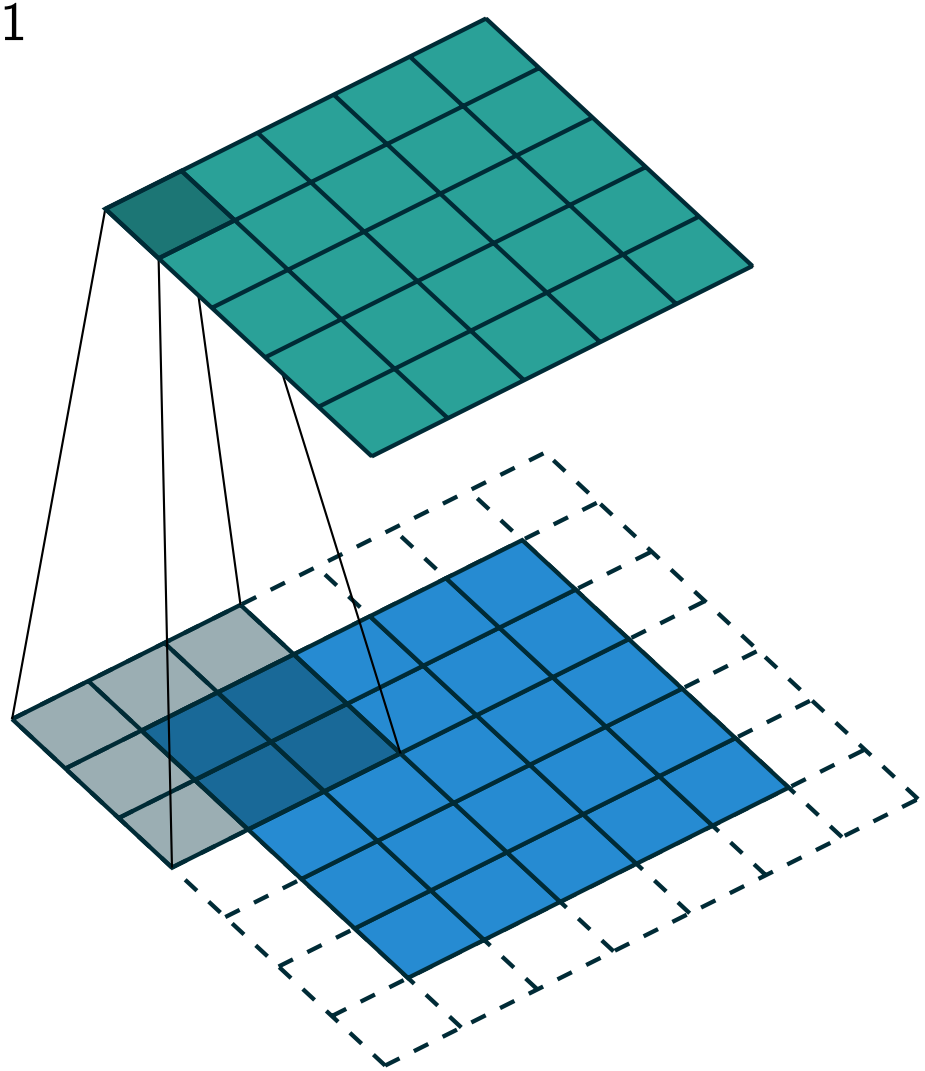
Convolução 2D: Exemplo



[15] 2026 — vdumoulin — *vdumoulin/conv_arithmetic*

Padding

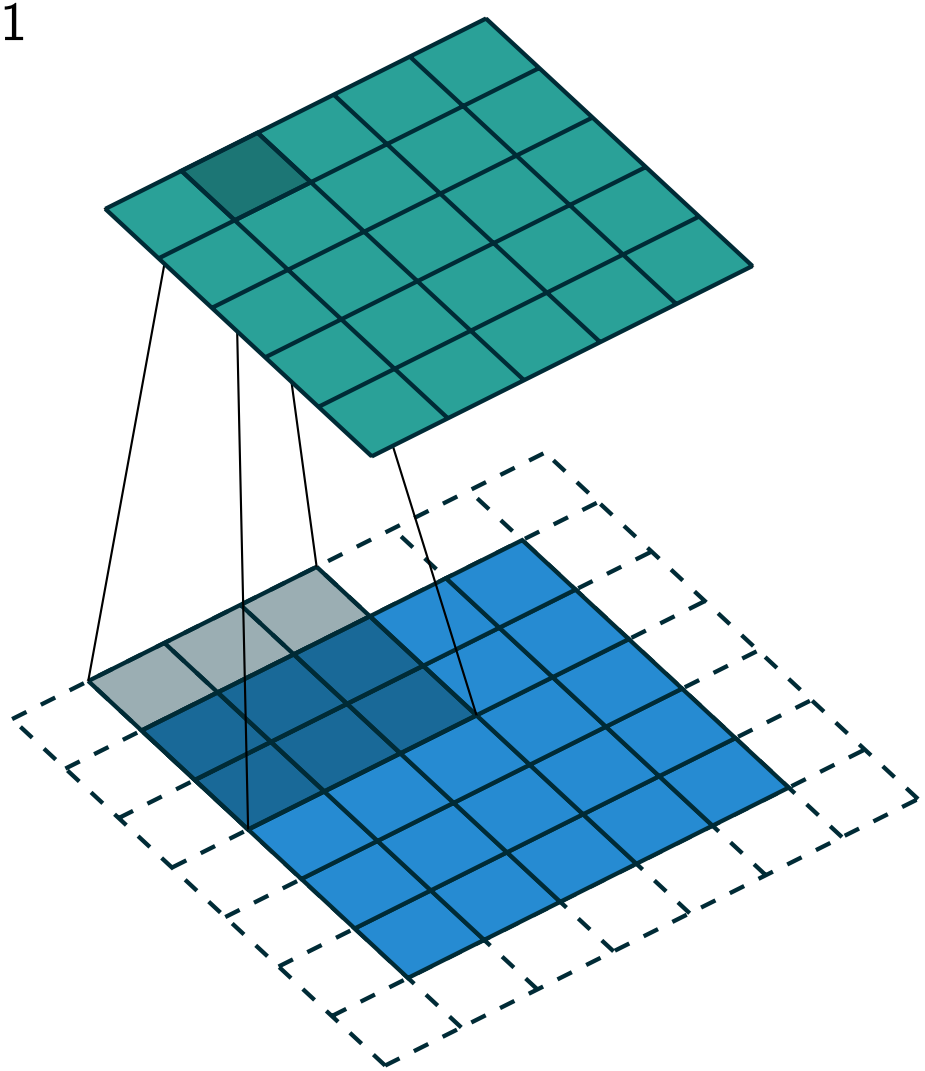
- padding $= 1$



[15] 2026 — vdumoulin — *vdumoulin/conv_arithmetic*

Padding

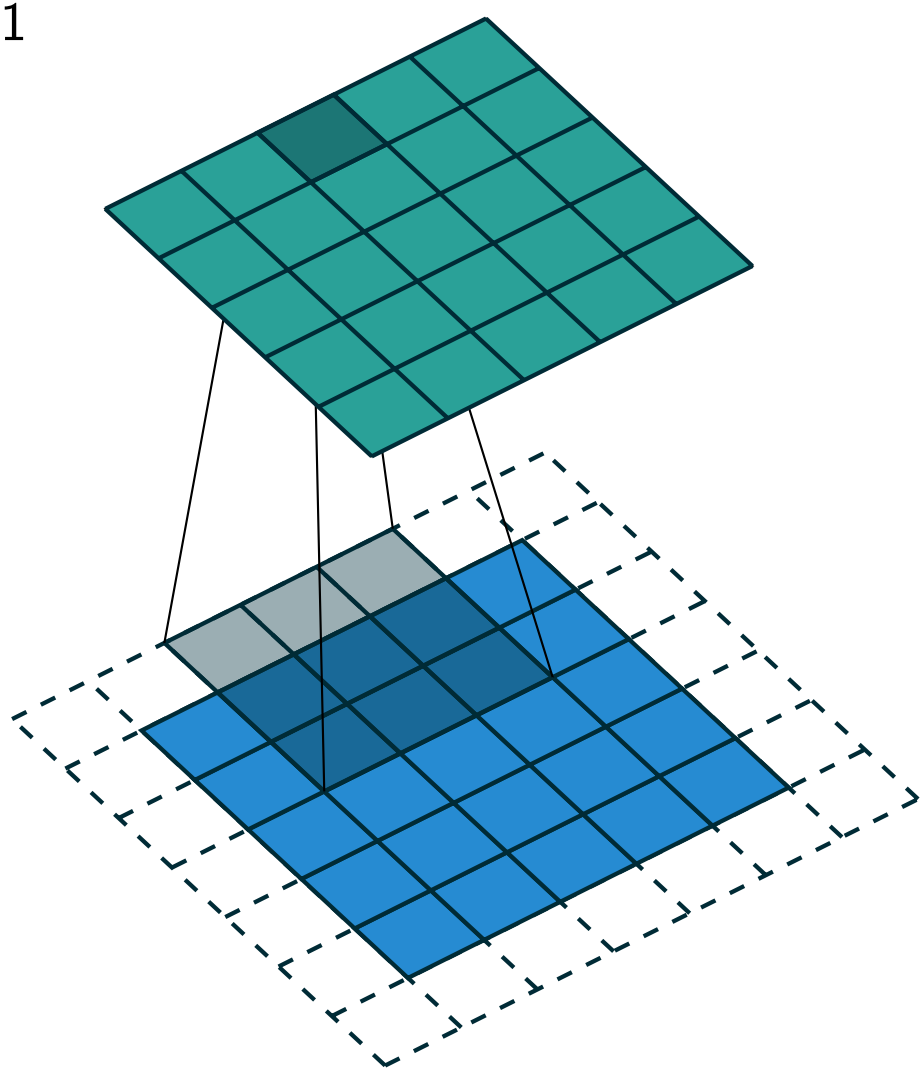
- padding $= 1$



[15] 2026 — vdumoulin — *vdumoulin/conv_arithmetic*

Padding

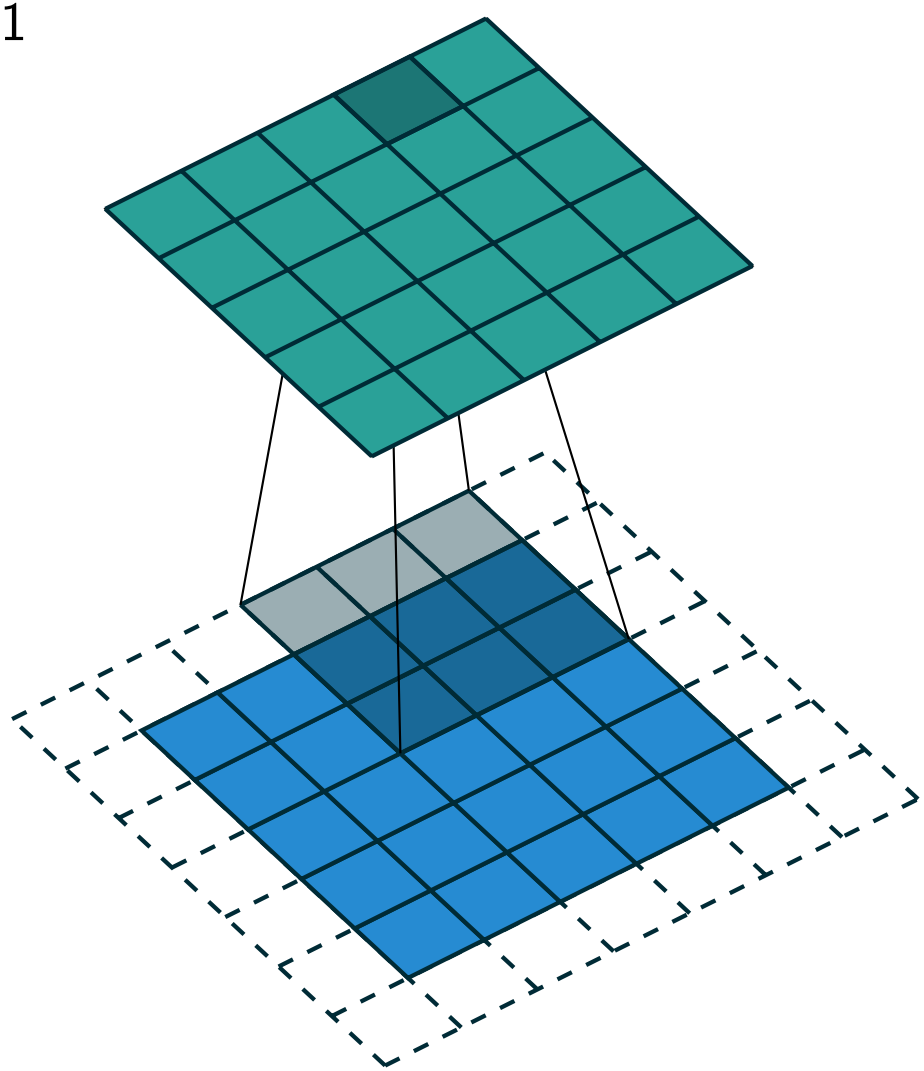
- padding $= 1$



[15] 2026 — vdumoulin — *vdumoulin/conv_arithmetic*

Padding

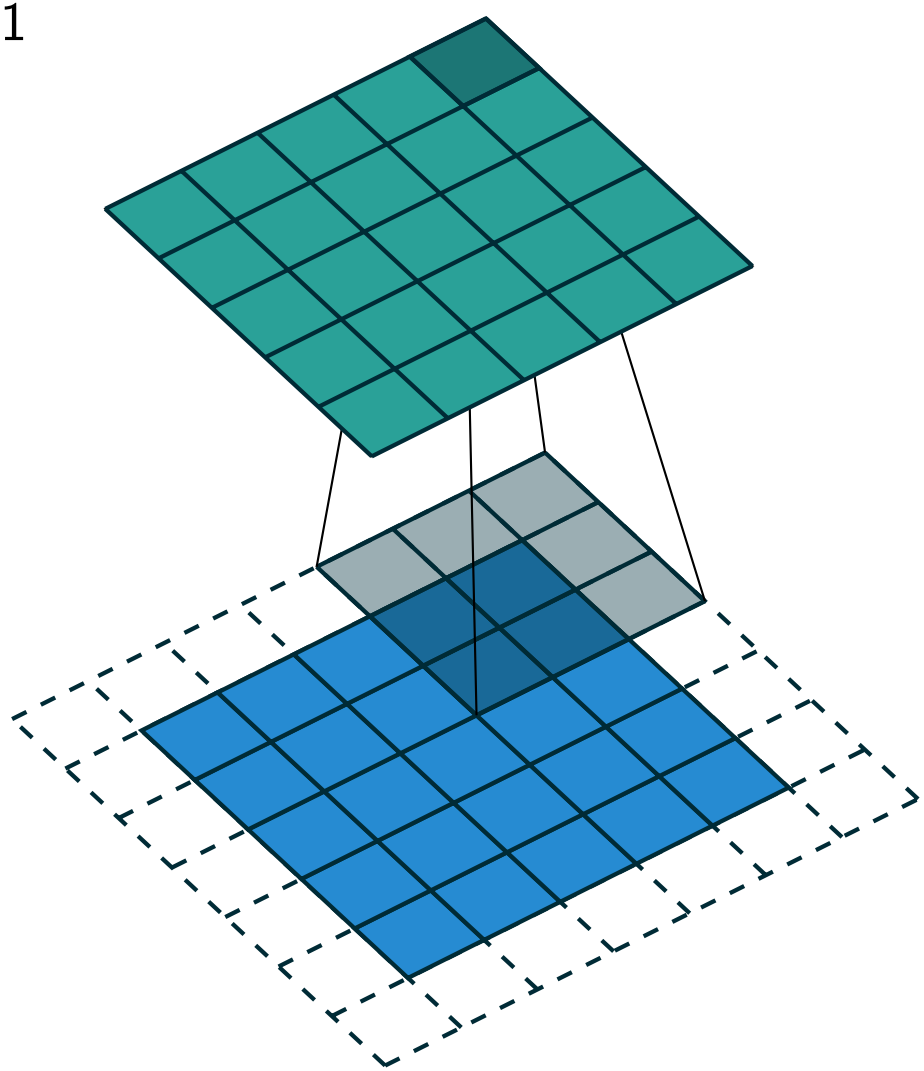
- padding $= 1$



[15] 2026 — vdumoulin — *vdumoulin/conv_arithmetic*

Padding

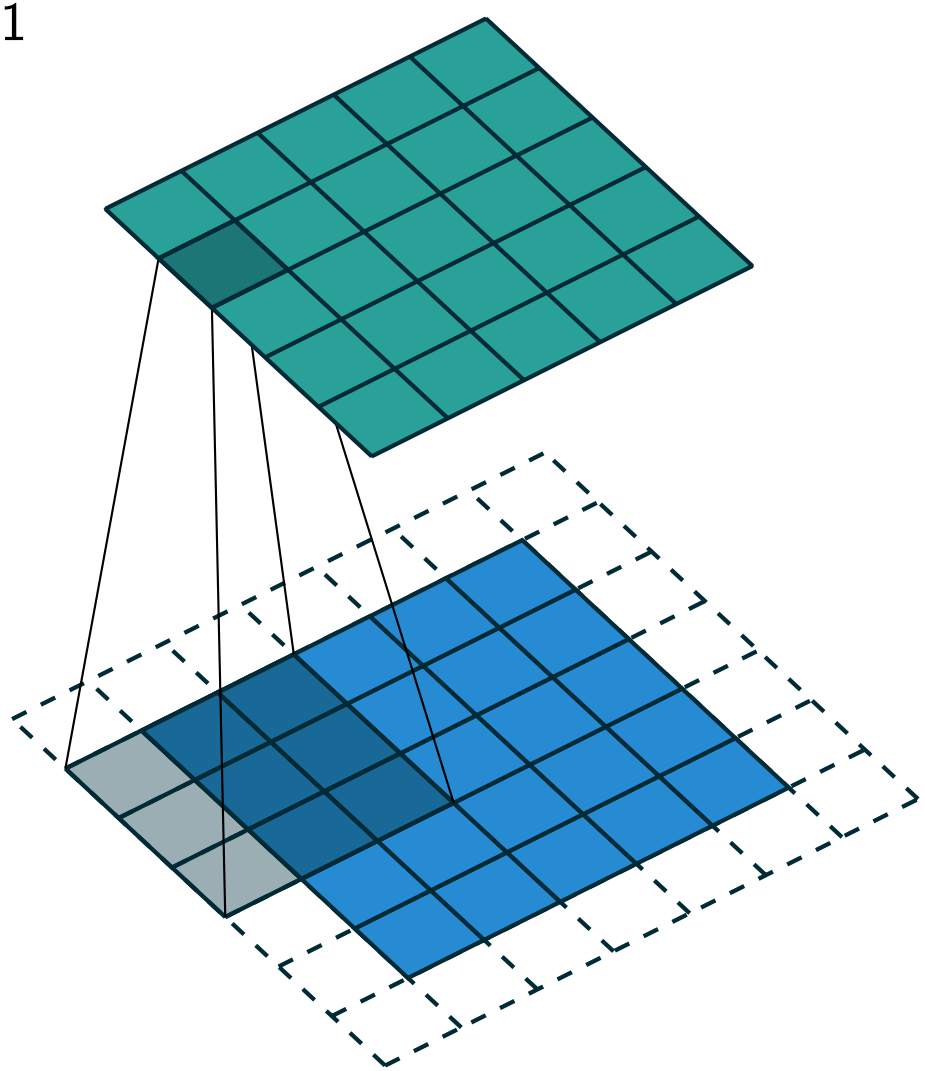
- padding $= 1$



[15] 2026 — vdumoulin — *vdumoulin/conv_arithmetic*

Padding

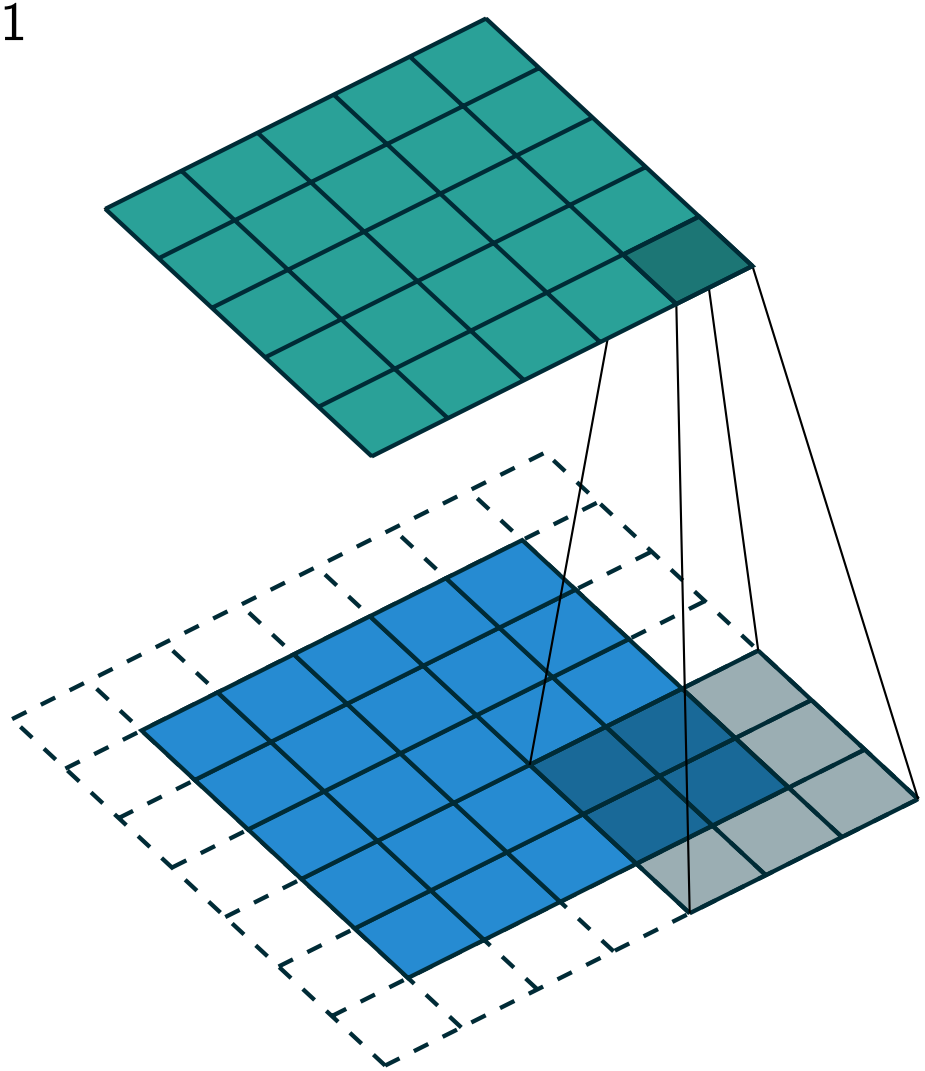
- padding $= 1$



[15] 2026 — vdumoulin — *vdumoulin/conv_arithmetic*

Padding

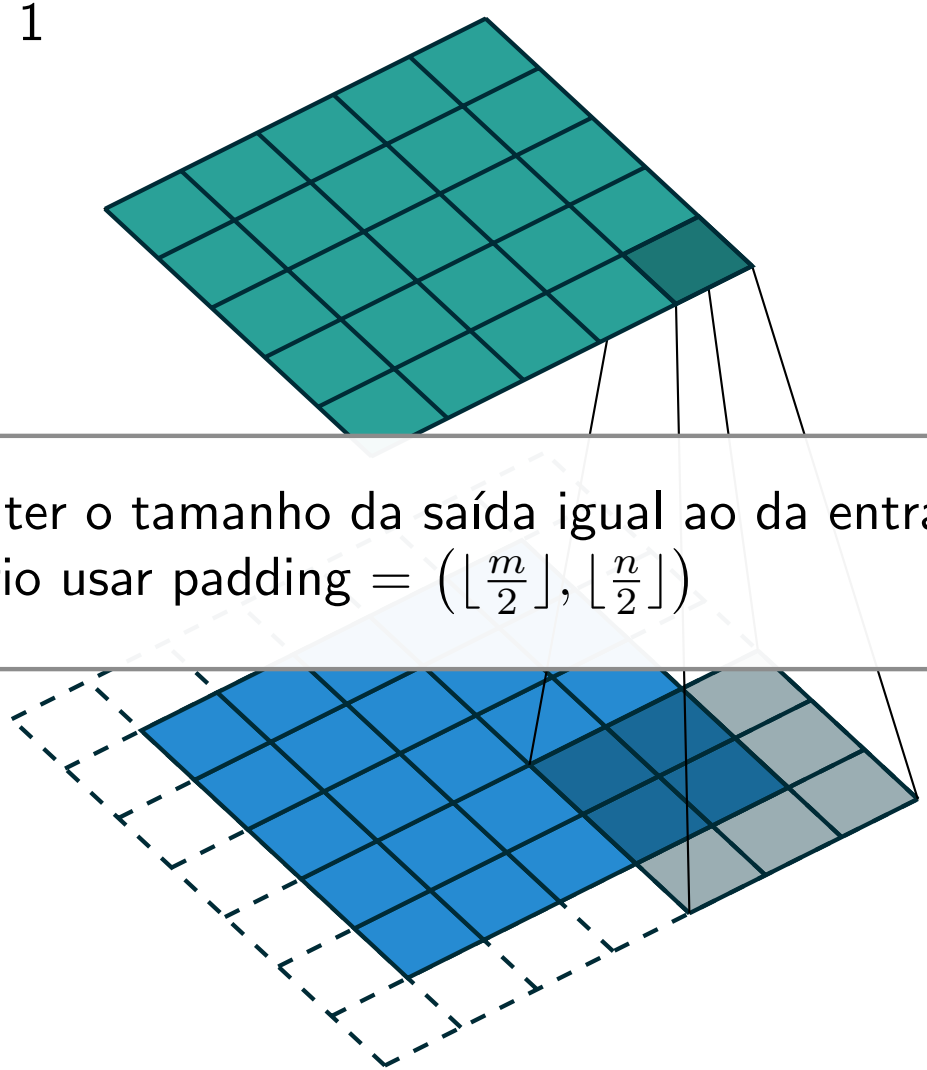
- padding $= 1$



[15] 2026 — vdumoulin — *vdumoulin/conv_arithmetic*

Padding

- padding $= 1$

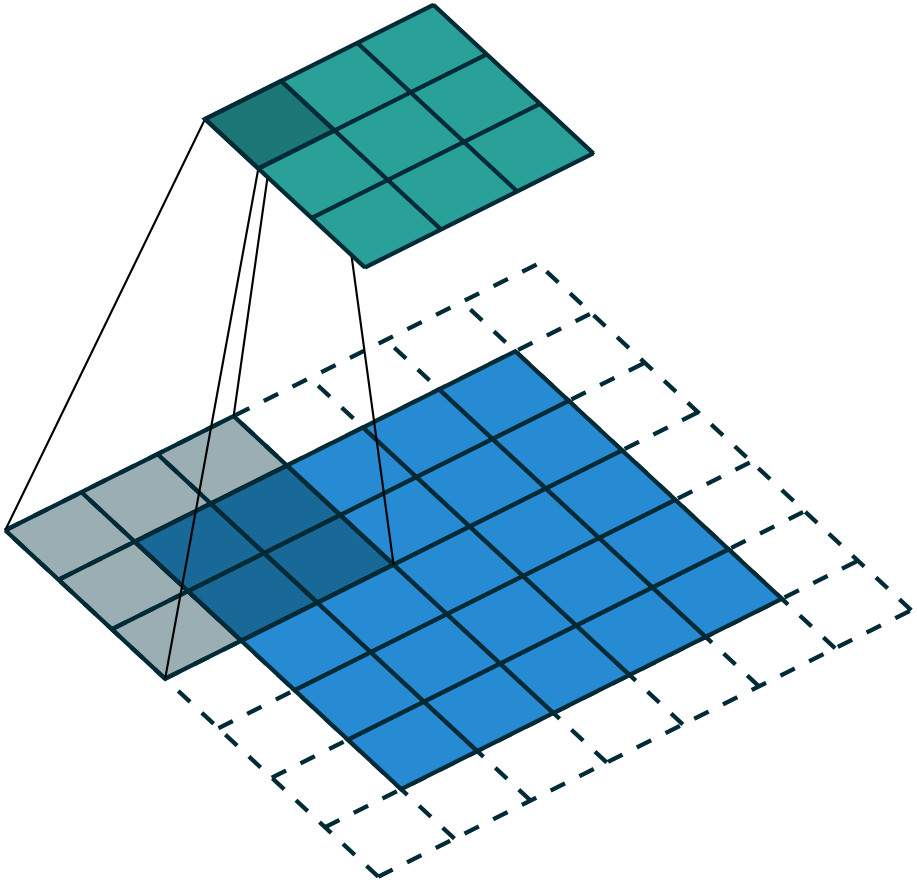


Para mater o tamanho da saída igual ao da entrada, é necessário usar padding $= \left(\lfloor \frac{m}{2} \rfloor, \lfloor \frac{n}{2} \rfloor\right)$

[15] 2026 — vdumoulin — *vdumoulin/conv_arithmetic*

Stride

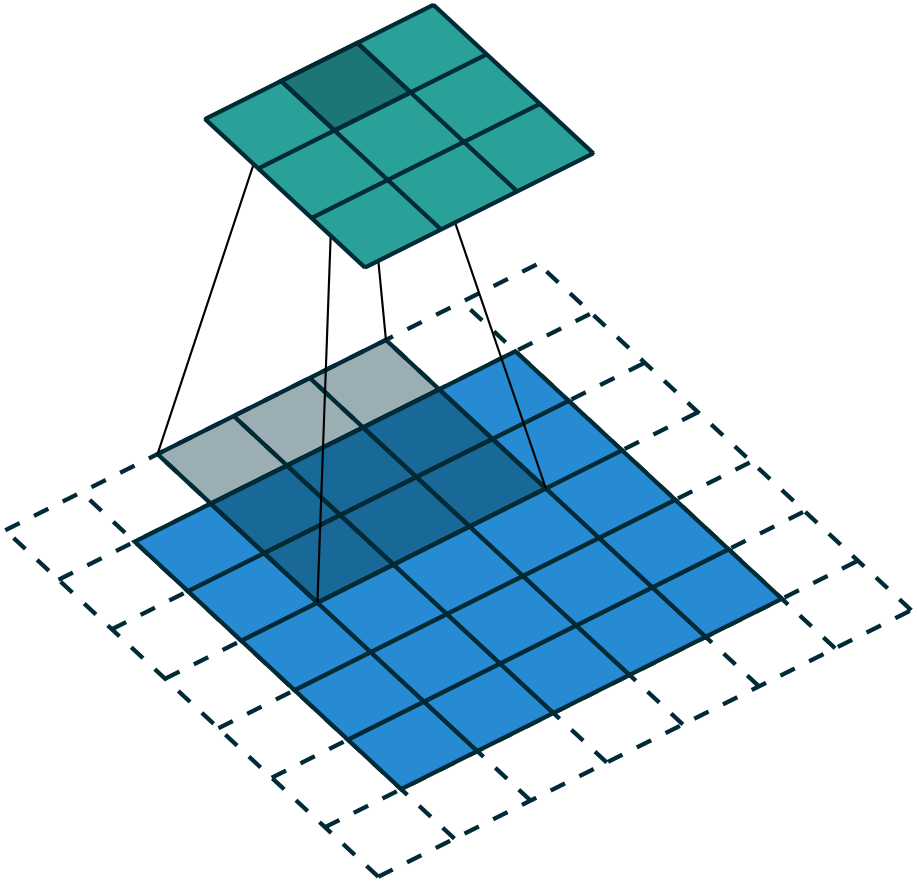
- stride $= 2$



[15] 2026 — vdumoulin — *vdumoulin/conv_arithmetic*

Stride

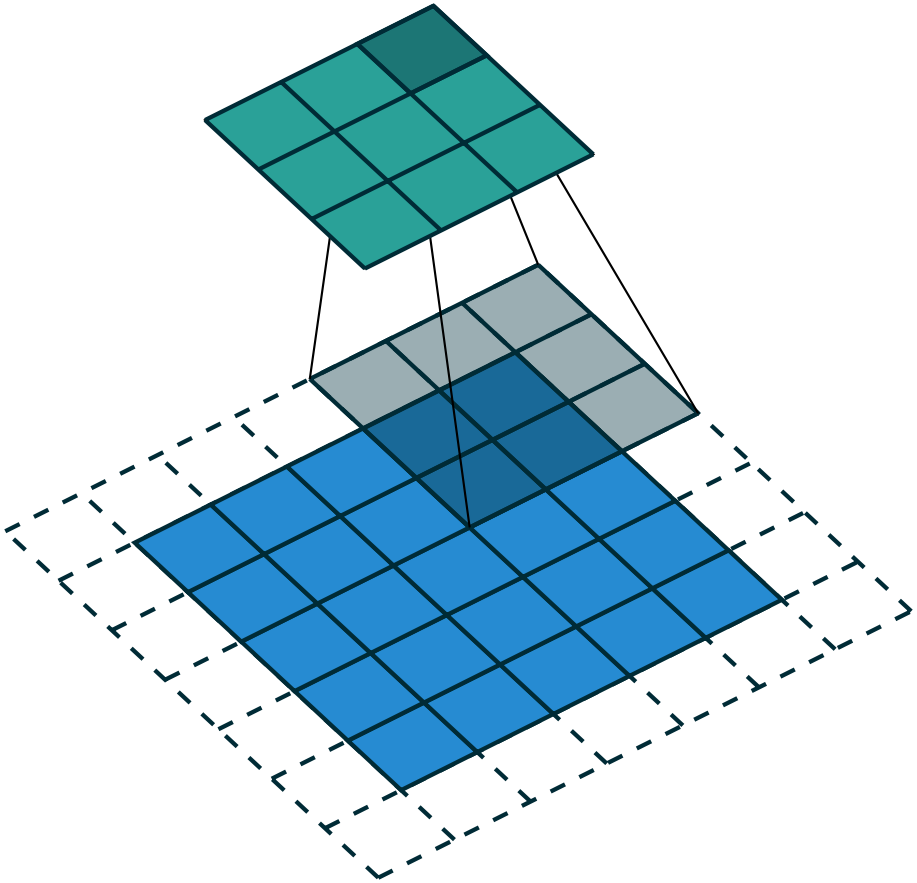
- stride $= 2$



[15] 2026 — vdumoulin — *vdumoulin/conv_arithmetic*

Stride

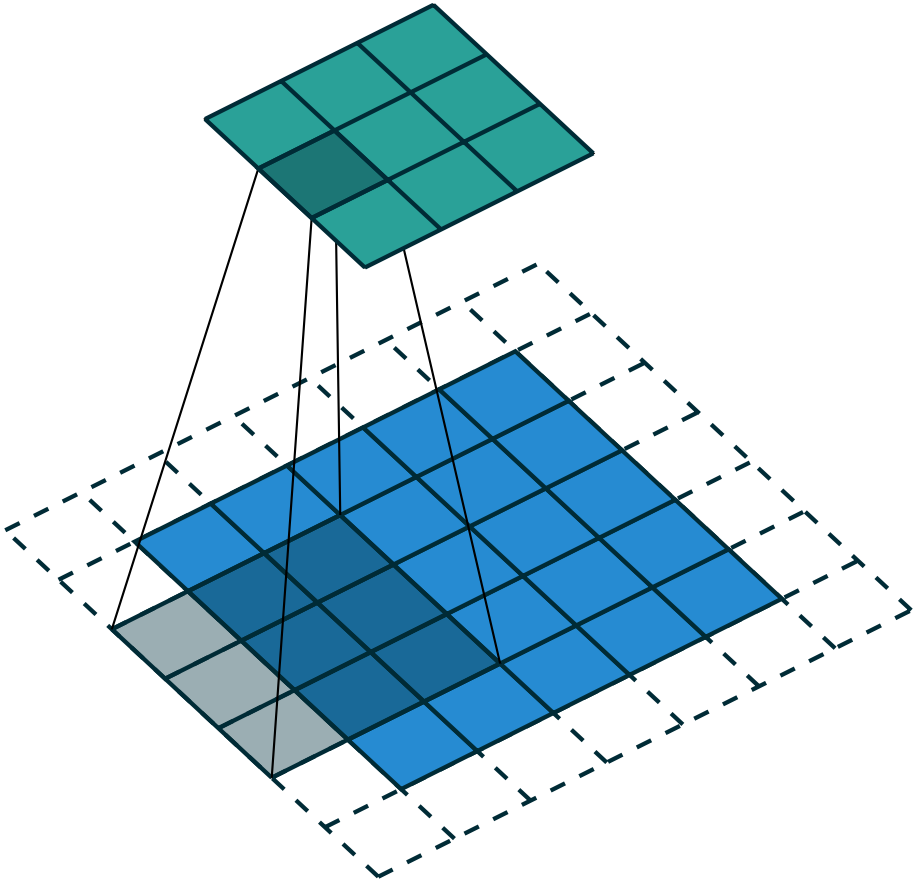
- stride $= 2$



[15] 2026 — vdumoulin — *vdumoulin/conv_arithmetic*

Stride

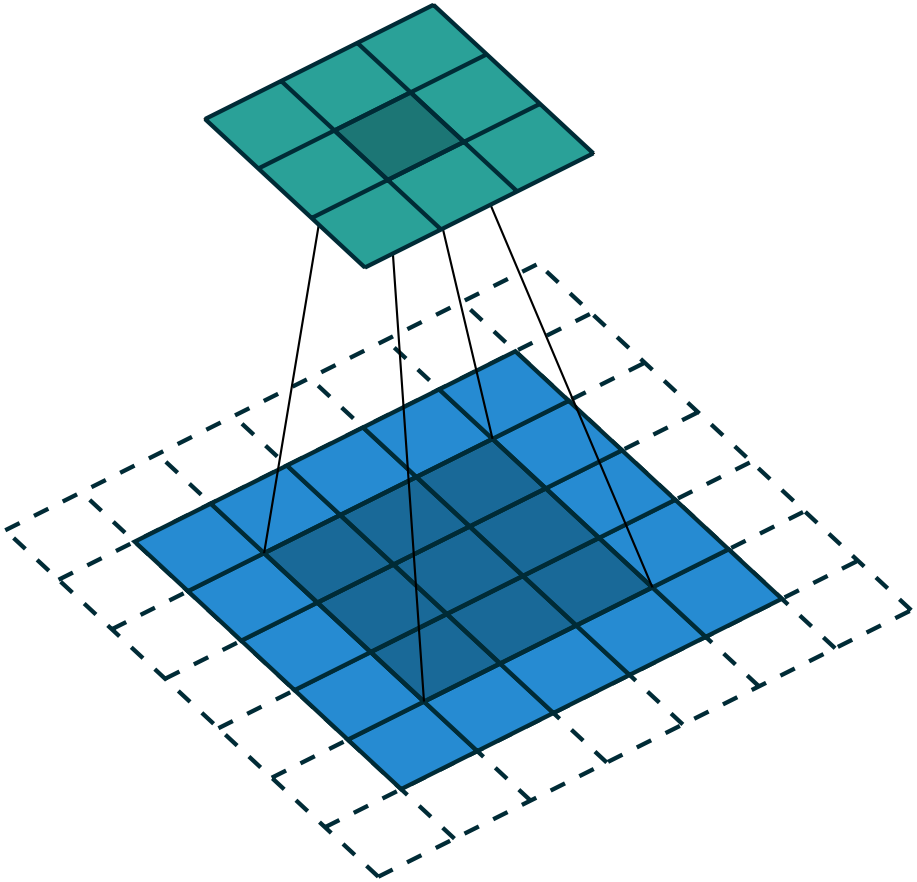
- stride $= 2$



[15] 2026 — vdumoulin — *vdumoulin/conv_arithmetic*

Stride

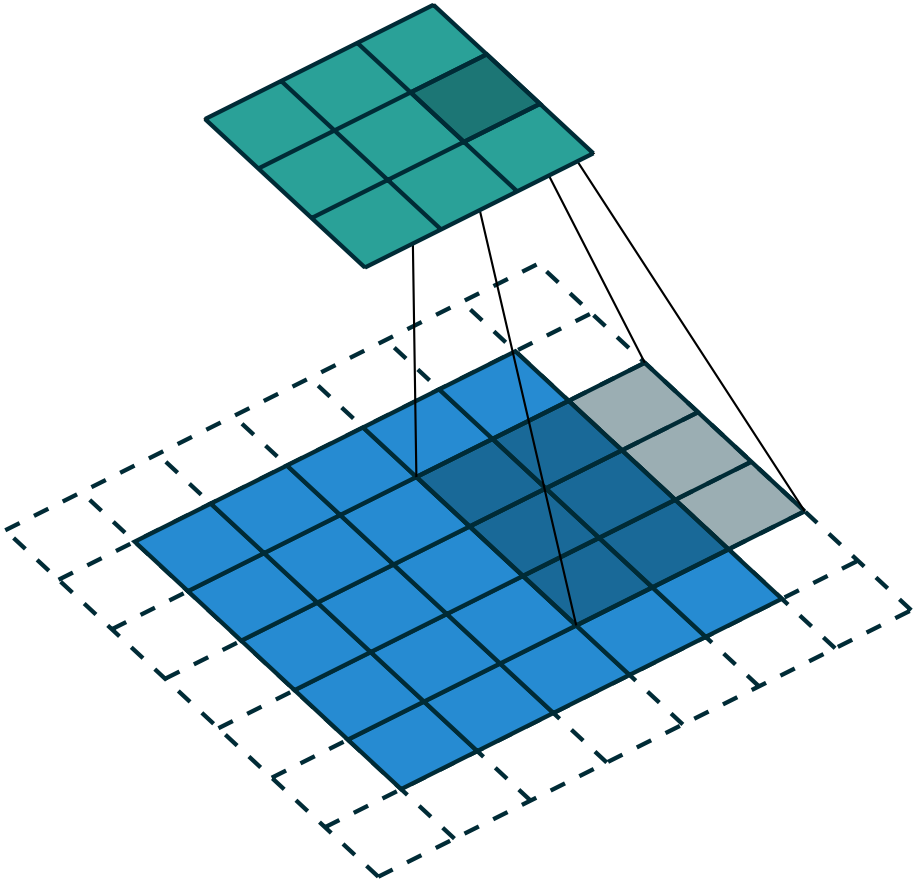
- stride $= 2$



[15] 2026 — vdumoulin — *vdumoulin/conv_arithmetic*

Stride

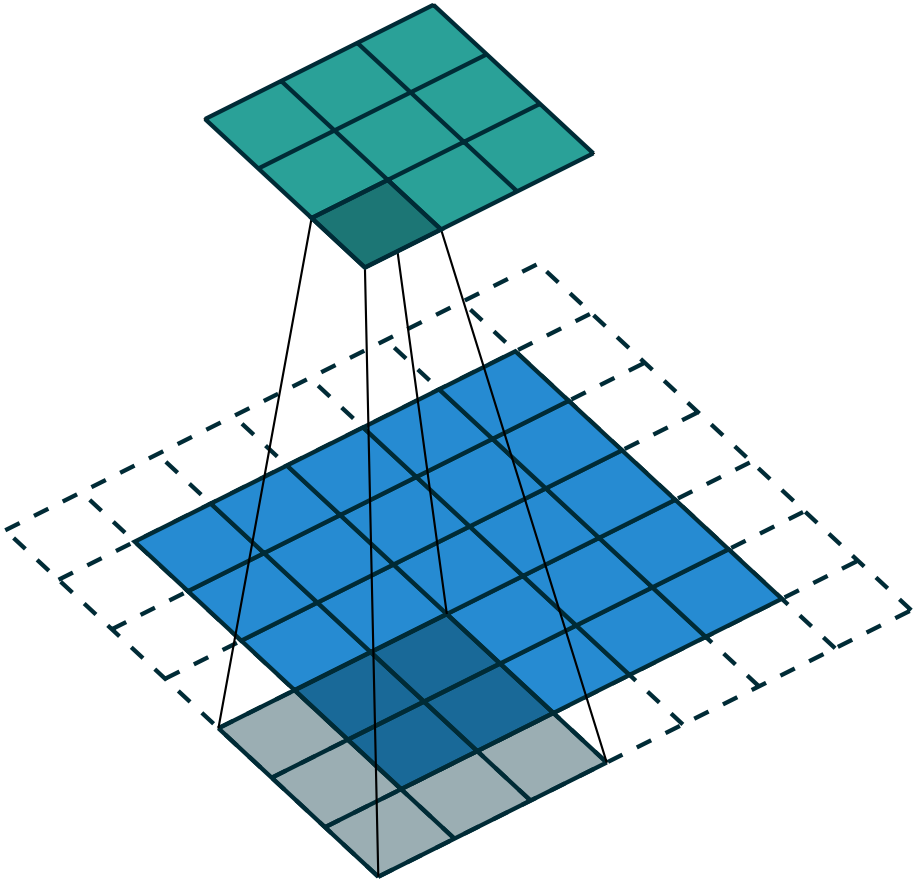
- stride $= 2$



[15] 2026 — vdumoulin — *vdumoulin/conv_arithmetic*

Stride

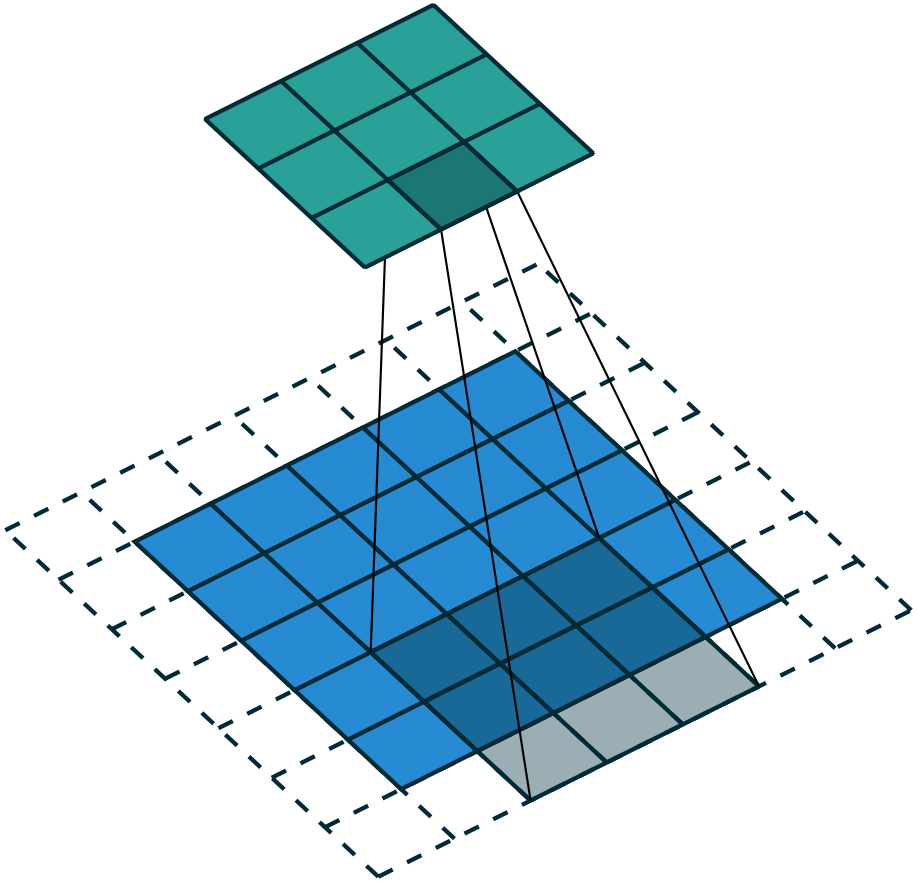
- stride $= 2$



[15] 2026 — vdumoulin — *vdumoulin/conv_arithmetic*

Stride

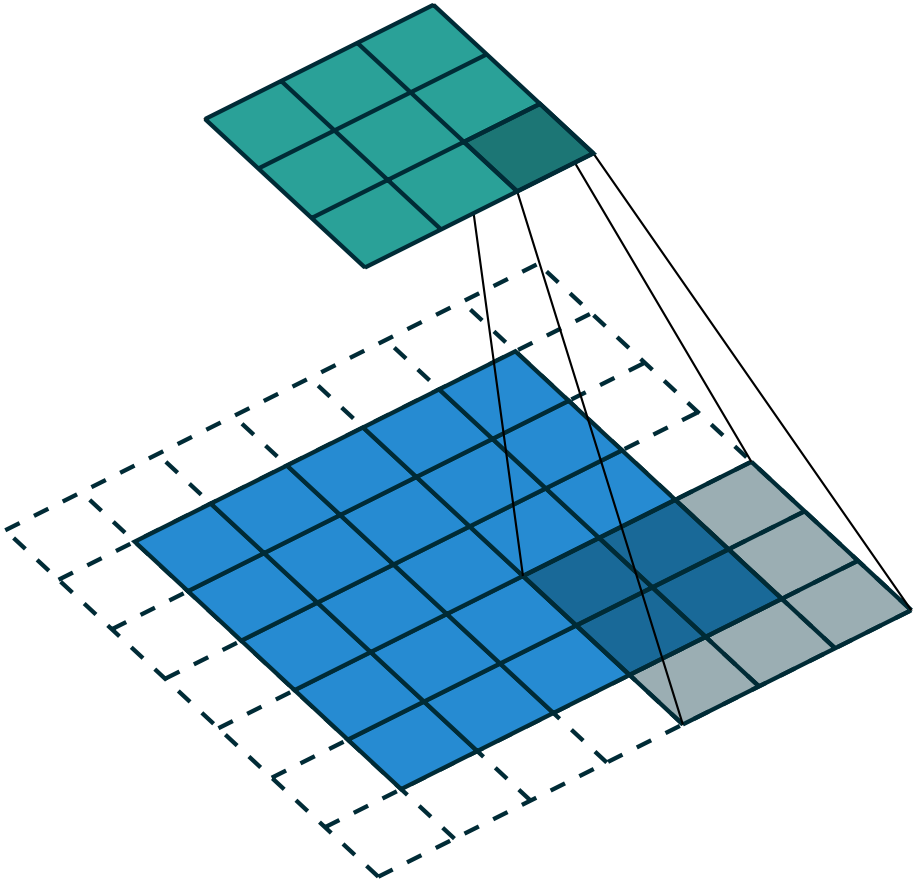
- stride $= 2$



[15] 2026 — vdumoulin — *vdumoulin/conv_arithmetic*

Stride

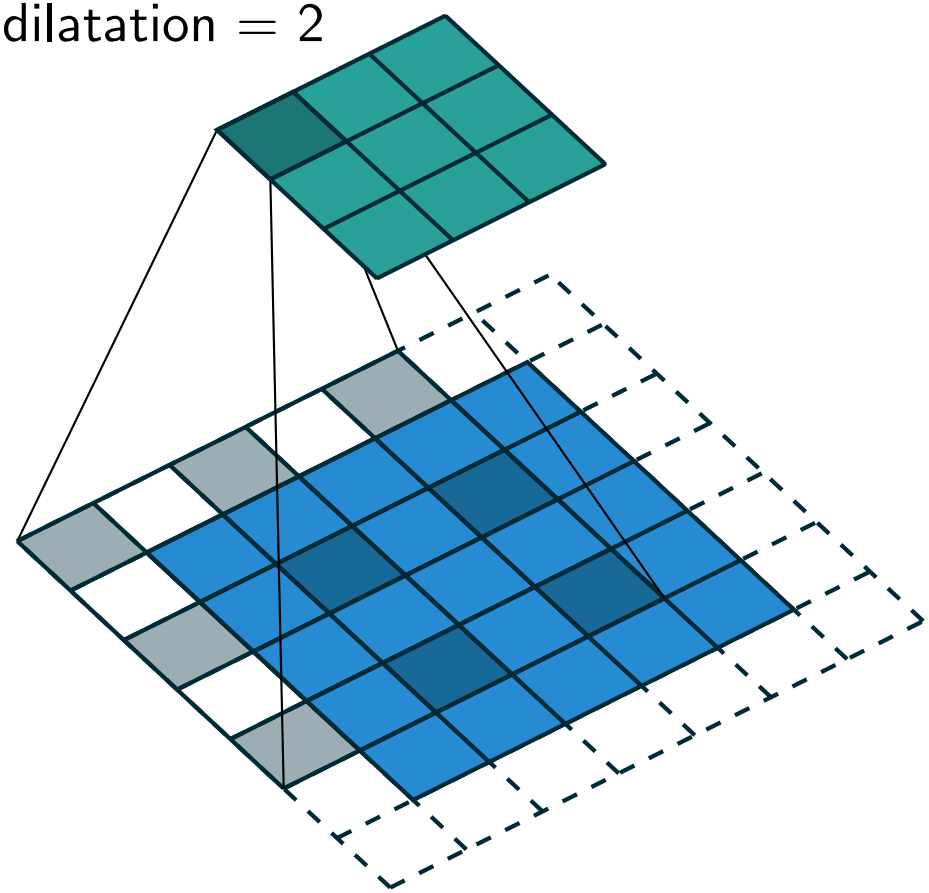
- stride $= 2$



[15] 2026 — vdumoulin — *vdumoulin/conv_arithmetic*

Dilatation

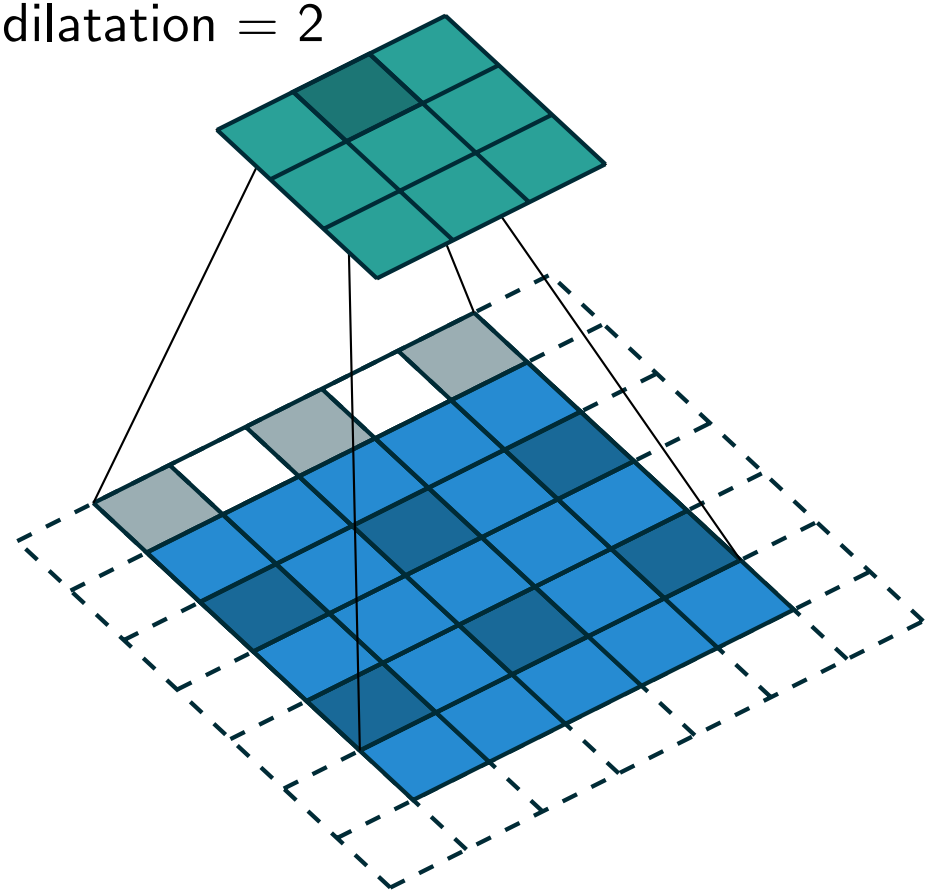
- stride = 1, dilatation = 2



[15] 2026 — vdumoulin — *vdumoulin/conv_arithmetic*

Dilatation

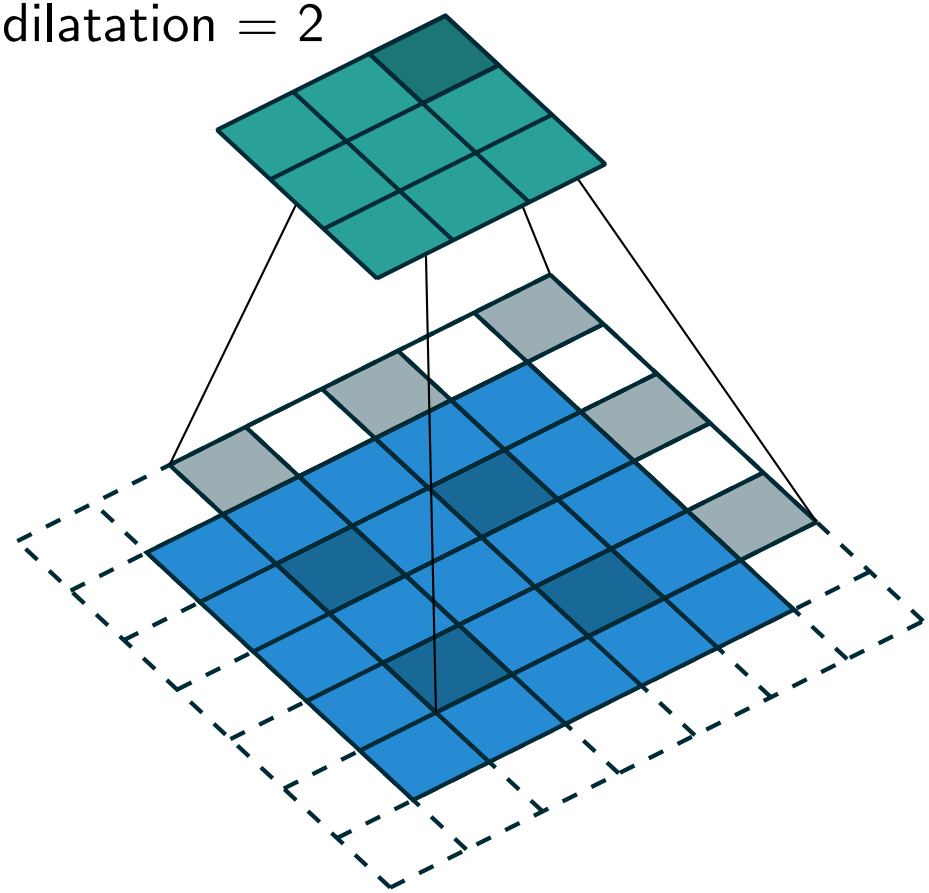
- stride = 1, dilatation = 2



[15] 2026 — vdumoulin — *vdumoulin/conv_arithmetic*

Dilatation

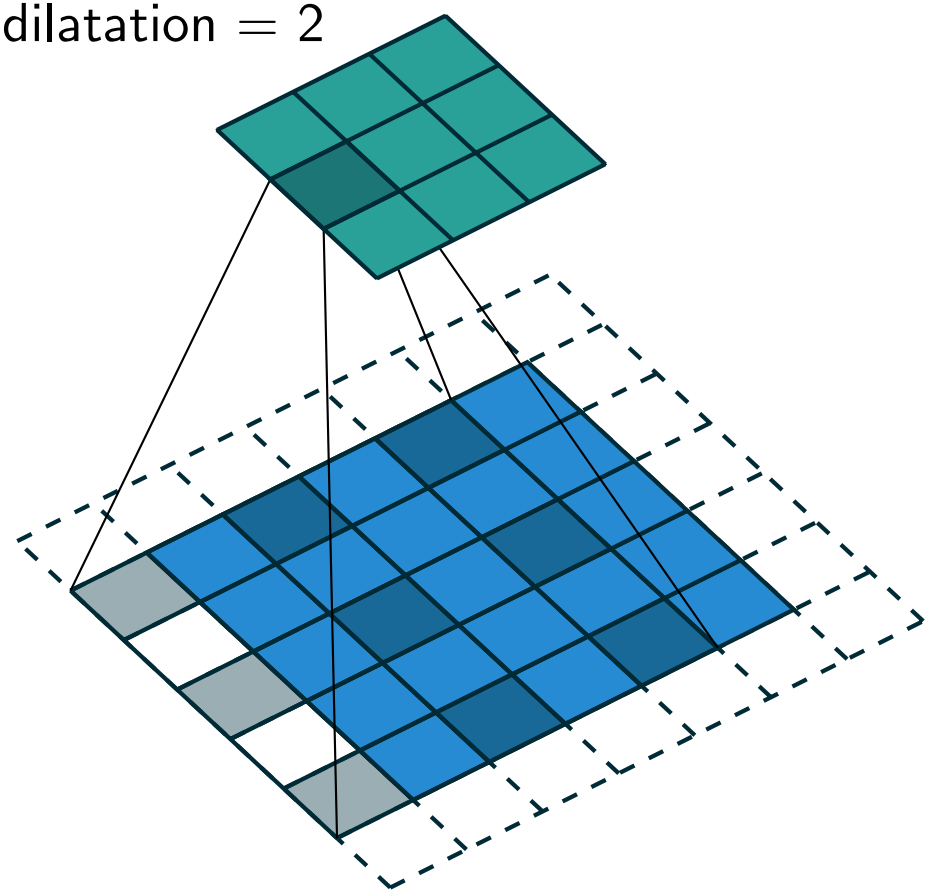
- stride = 1, dilatation = 2



[15] 2026 — vdumoulin — *vdumoulin/conv_arithmetic*

Dilatation

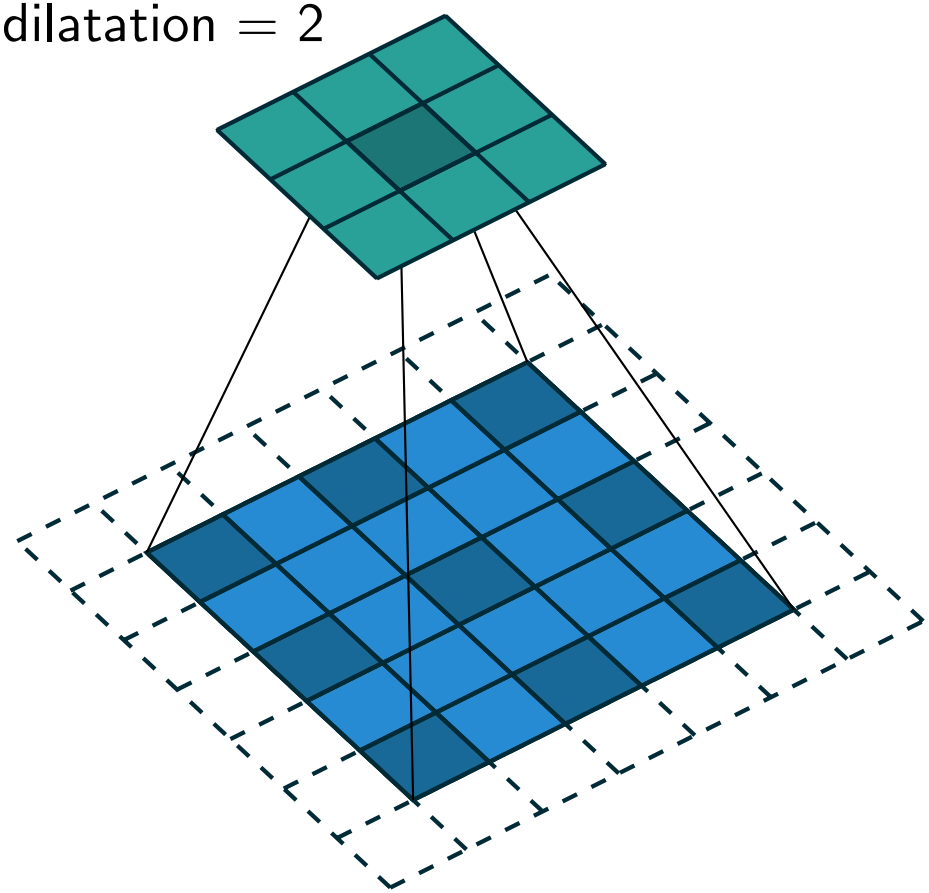
- stride = 1, dilatation = 2



[15] 2026 — vdumoulin — *vdumoulin/conv_arithmetic*

Dilatation

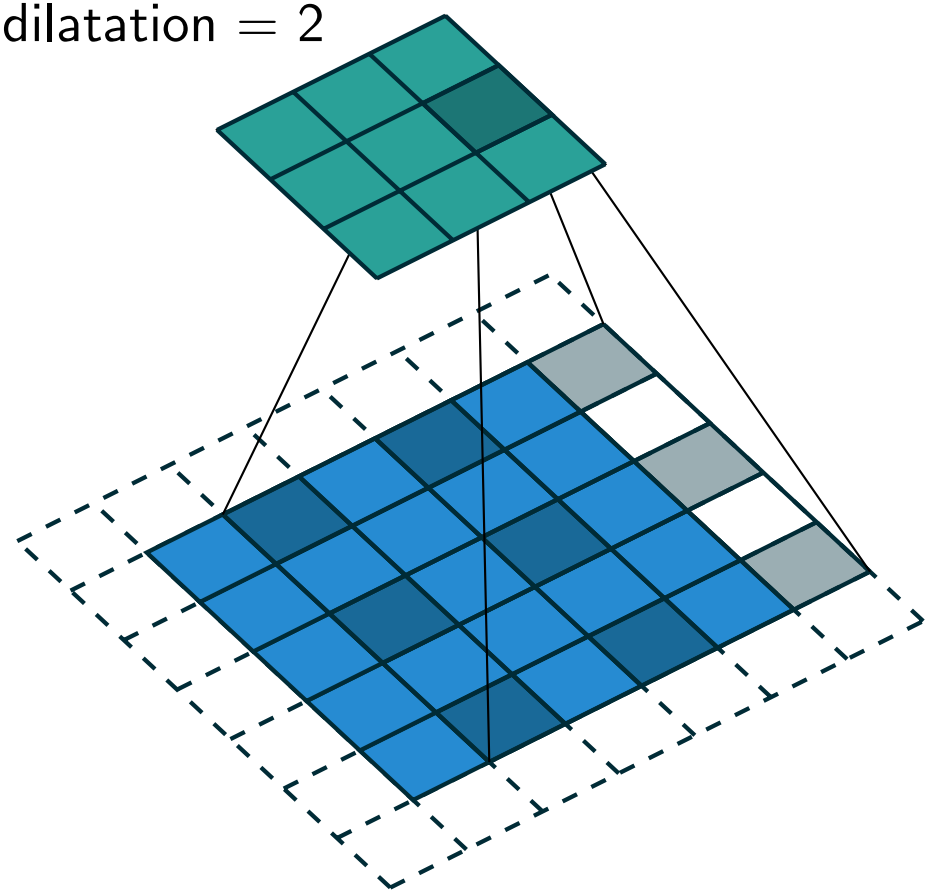
- stride = 1, dilatation = 2



[15] 2026 — vdumoulin — *vdumoulin/conv_arithmetic*

Dilatation

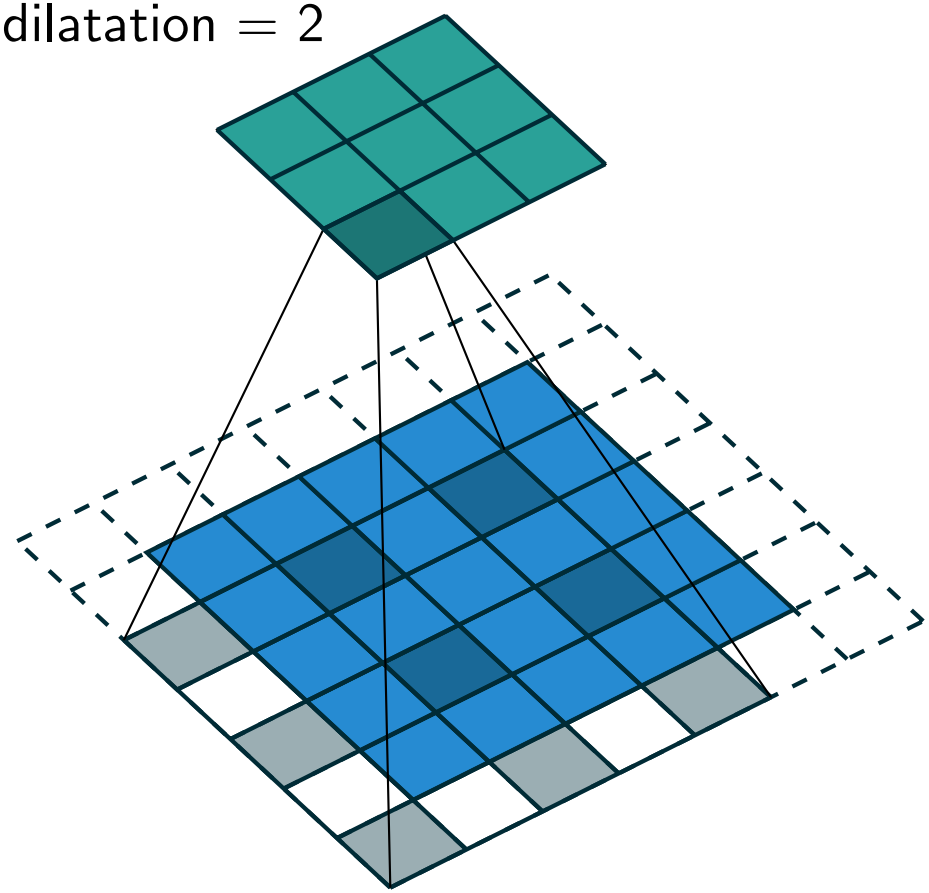
- stride = 1, dilatation = 2



[15] 2026 — vdumoulin — *vdumoulin/conv_arithmetic*

Dilatation

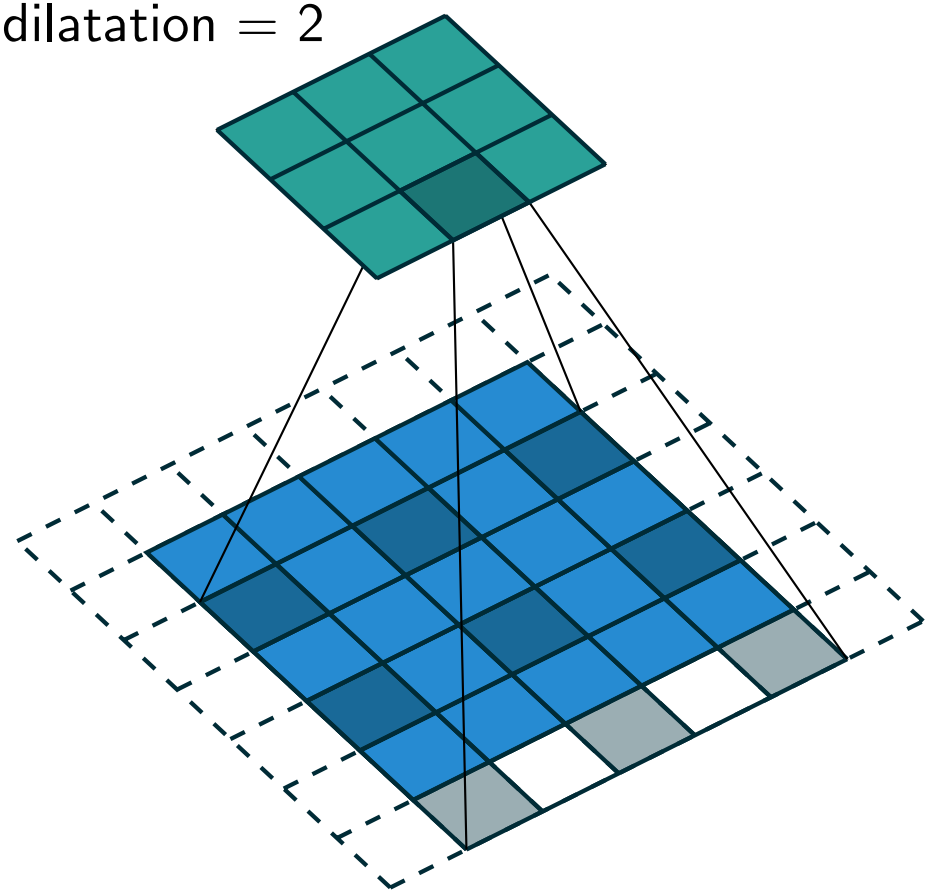
- stride = 1, dilatation = 2



[15] 2026 — vdumoulin — *vdumoulin/conv_arithmetic*

Dilatation

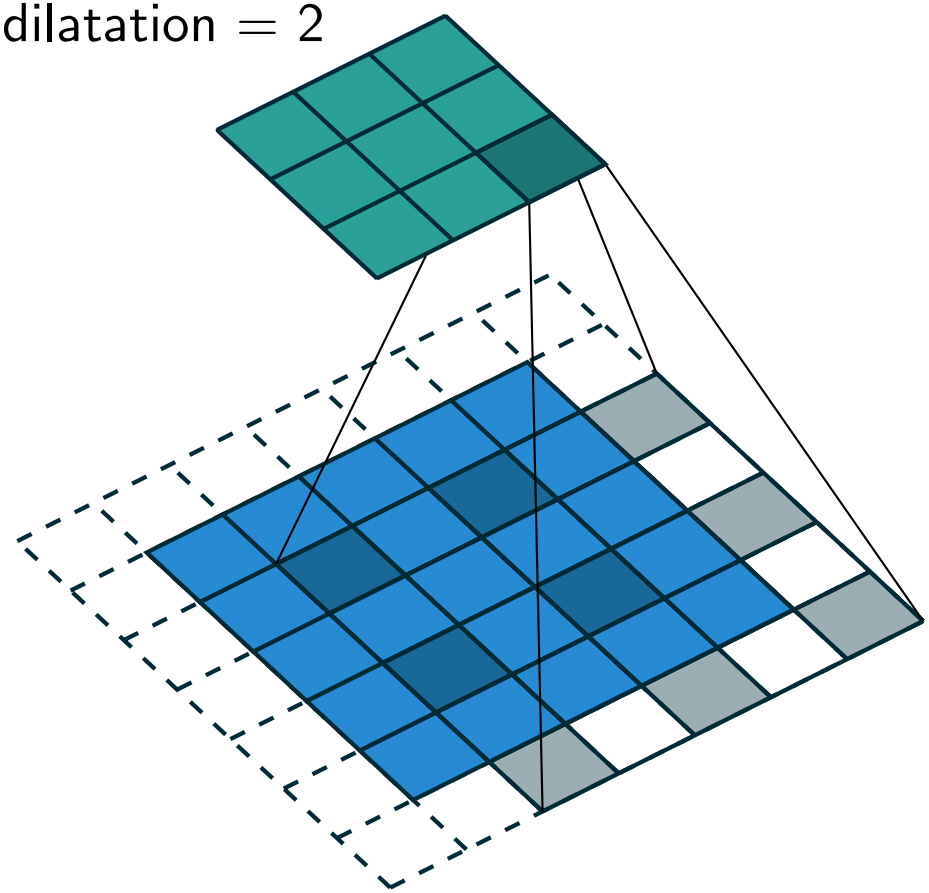
- stride = 1, dilatation = 2



[15] 2026 — vdumoulin — *vdumoulin/conv_arithmetic*

Dilatation

- stride = 1, dilatation = 2



[15] 2026 — vdumoulin — *vdumoulin/conv_arithmetic*

Qual é o tamanho da saída?

$$L_{out} = \left\lfloor \frac{L_{in} + 2 \times \text{padding} - \text{dilatation} \times (\text{kernel_size} - 1) - 1}{\text{stride}} + 1 \right\rfloor$$

[3] — *Conv2d — PyTorch 2.6 documentation*

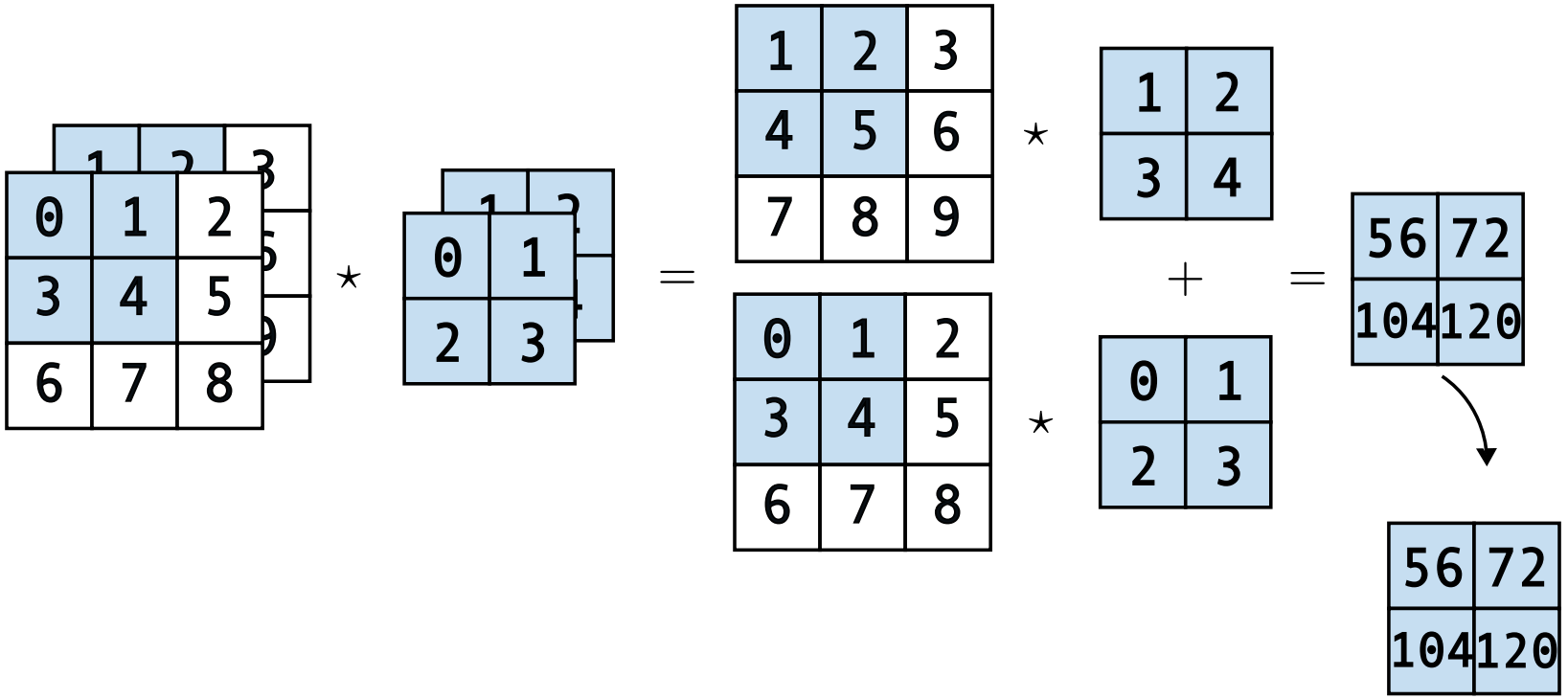
Convolução com Multiplos canais

C canais de entrada e 1 de saída

- $I = [I_{i,j,c}]$, onde $i \in [0, H-1]$, $j \in [0, W-1]$ e $c \in [0, C-1]$
- $K = [K_{u,v,c}]$, onde $u \in [0, m-1]$, $v \in [0, n-1]$ e $c \in [0, C-1]$

$$
\begin{aligned}
[I \star K]_{i,j} &= \sum_{c=0}^{C-1} \sum_{i=0}^{H-1} \sum_{j=0}^{W-1} I_{i,j,c} K_{i-u,j-v,c}, \text{ ou} \\
&= \sum_{c=0}^{C-1} \sum_{u=0}^{m-1} \sum_{v=0}^{n-1} I_{i+u,j+v,c} K_{u,v,c}
\end{aligned}
$$

Convolução com Multiplos canais: Exemplo



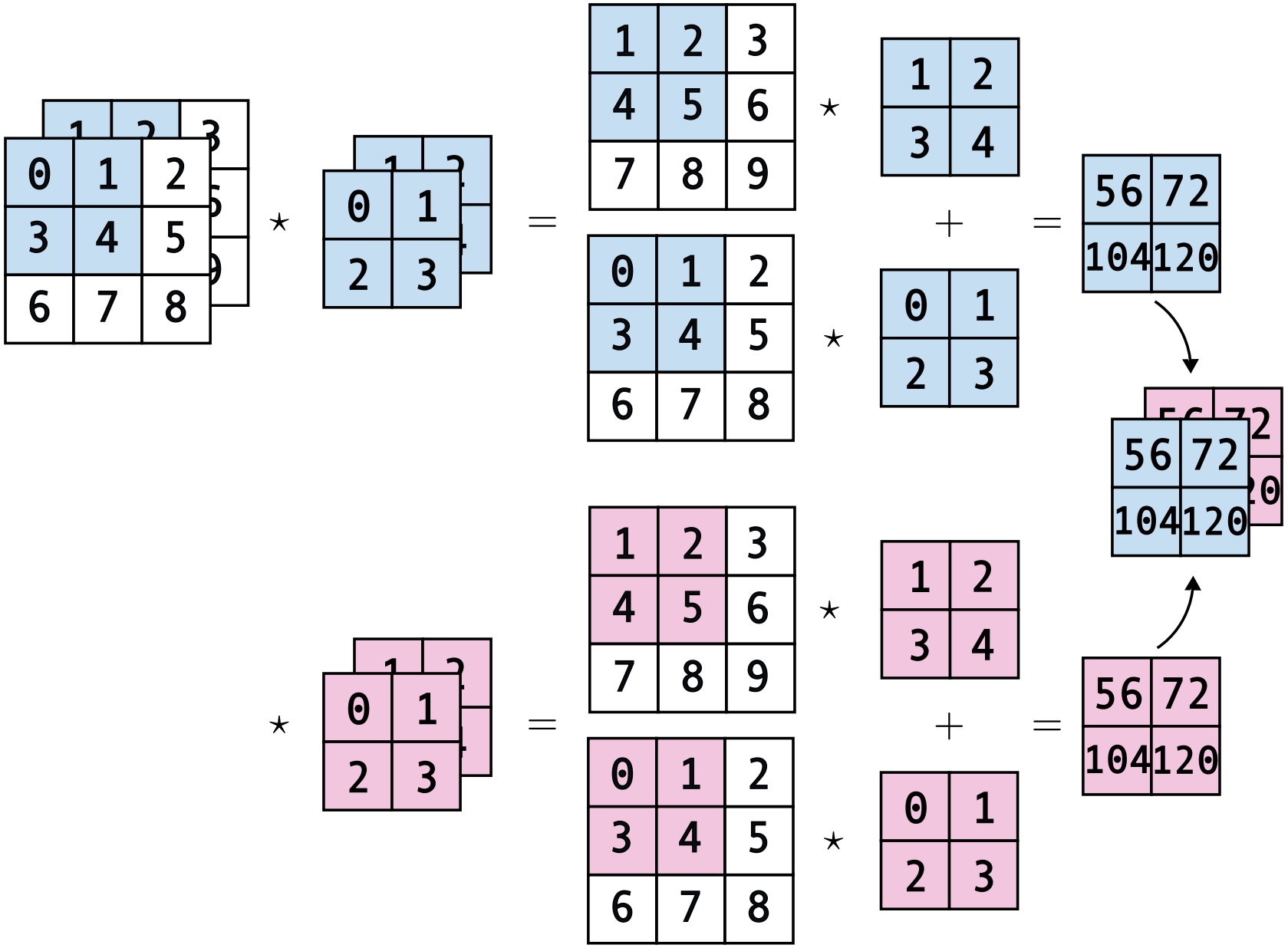
Convolução com Multiplos canais

C canais de entrada e D de saída

- $I = [I_{i,j,c}]$, onde $i \in [0, H-1]$, $j \in [0, W-1]$ e $c \in [0, C-1]$
- $K = [K_{u,v,c,d}]$, onde $u \in [0, m-1]$, $v \in [0, n-1]$, $c \in [0, C-1]$ e $d \in [0, D-1]$

$$
\begin{aligned}
[I \star K]_{i,j,d} &= \sum_{c=0}^{C-1} \sum_{i=0}^{H-1} \sum_{j=0}^{W-1} I_{i,j,c} K_{i-u,j-v,c,d}, \text{ ou} \\
&= \sum_{c=0}^{C-1} \sum_{u=0}^{m-1} \sum_{v=0}^{n-1} I_{i+u,j+v,c} K_{u,v,c,d}
\end{aligned}
$$

Convolução com Multiplos canais: Exemplo



Receptive field

$K =$

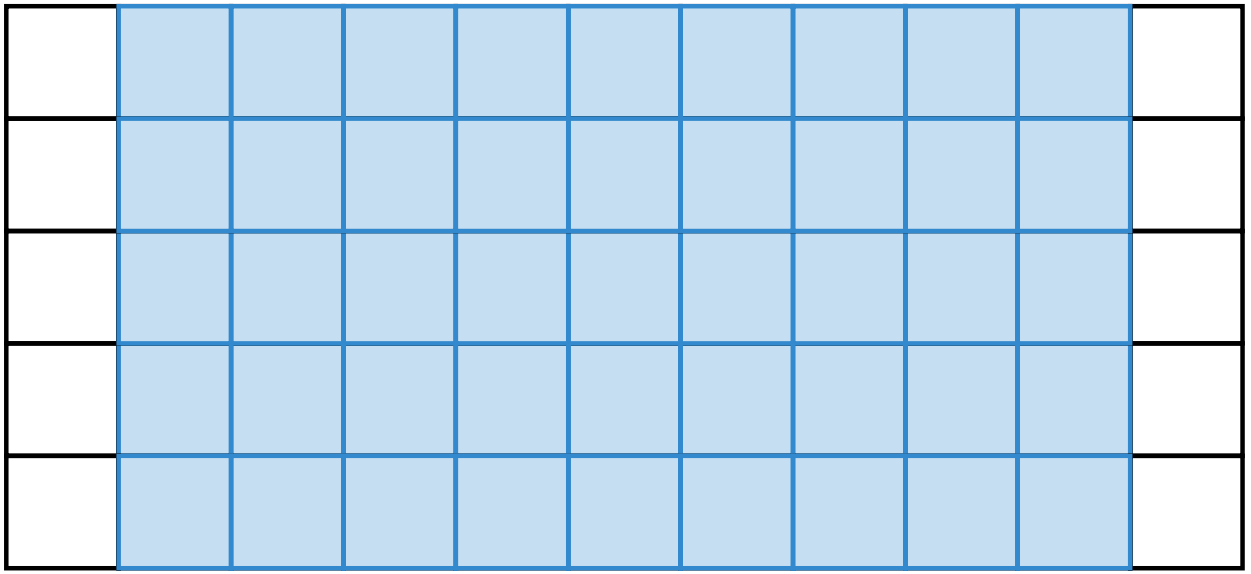
$I_1 = I =$

$I_2 = I_1 \star K =$

$I_3 = I_2 \star K =$

$I_4 = I_3 \star K =$

$I_5 = I_4 \star K =$



Receptive field

$$K =$$

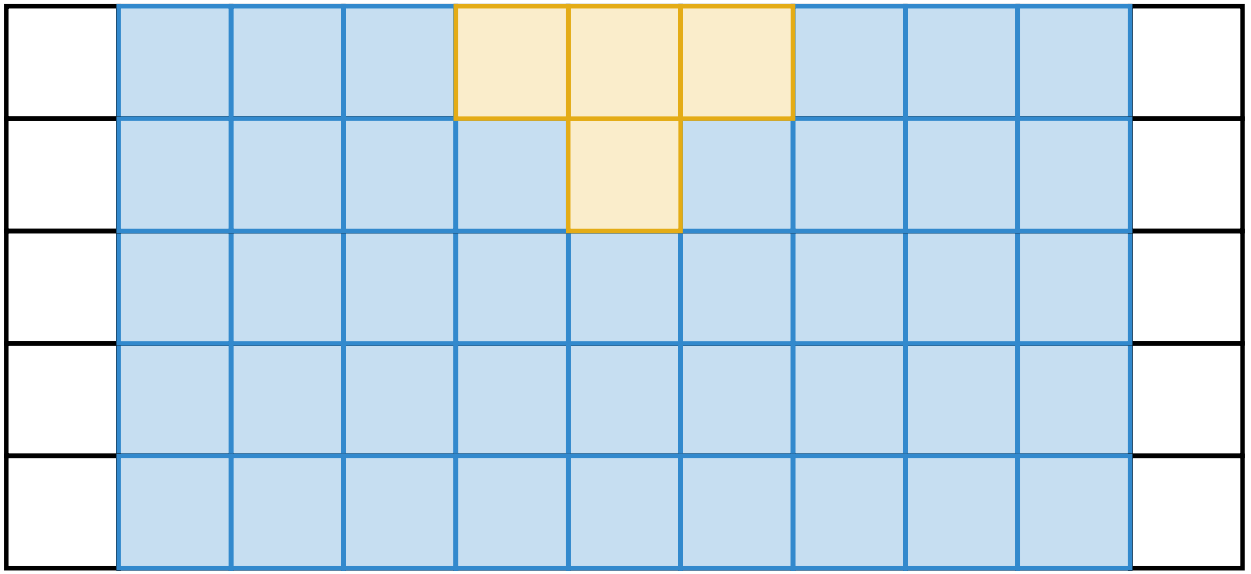
$$I_1 = I =$$

$$I_2 = I_1 \star K =$$

$$I_3 = I_2 \star K =$$

$$I_4 = I_3 \star K =$$

$$I_5 = I_4 \star K =$$



Receptive field

$K =$

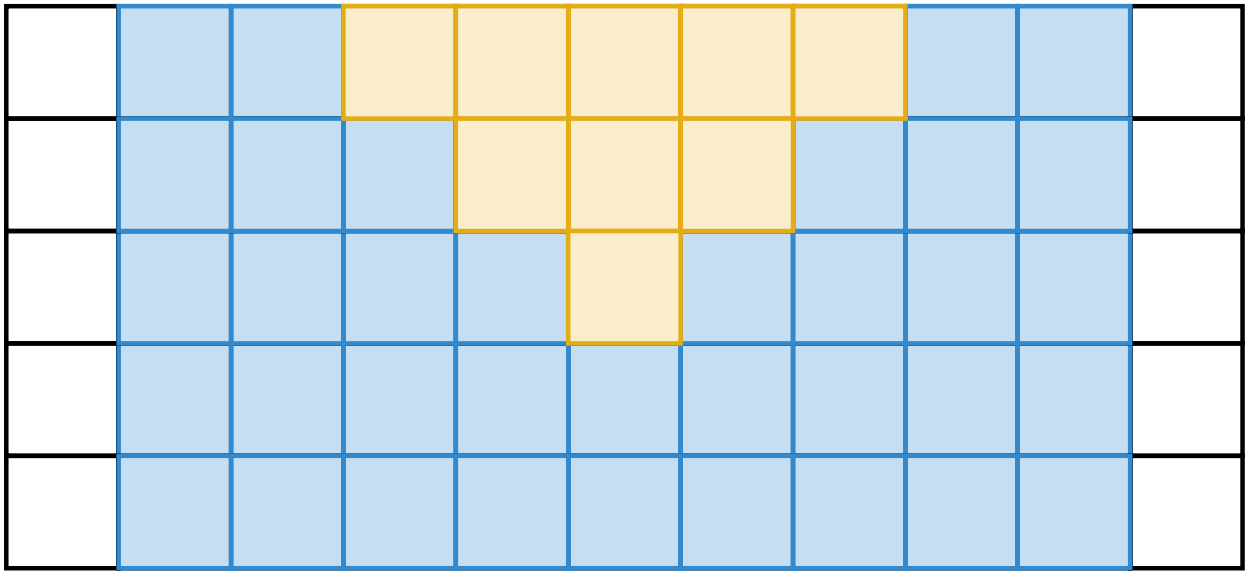
$I_1 = I =$

$I_2 = I_1 \star K =$

$I_3 = I_2 \star K =$

$I_4 = I_3 \star K =$

$I_5 = I_4 \star K =$



Receptive field

$K =$

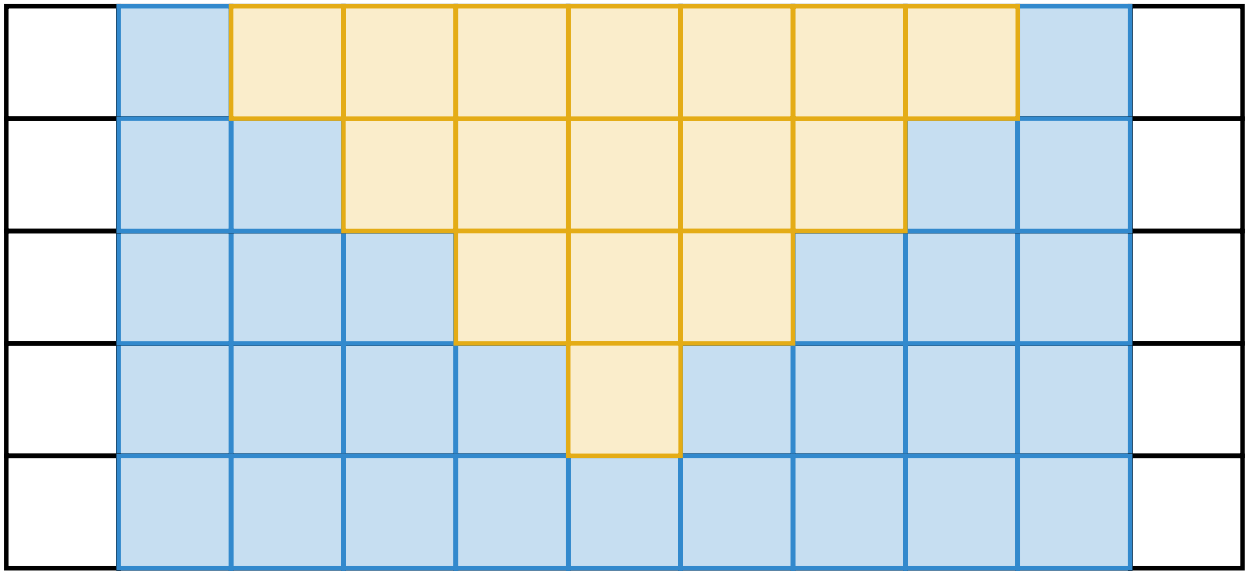
$I_1 = I =$

$I_2 = I_1 \star K =$

$I_3 = I_2 \star K =$

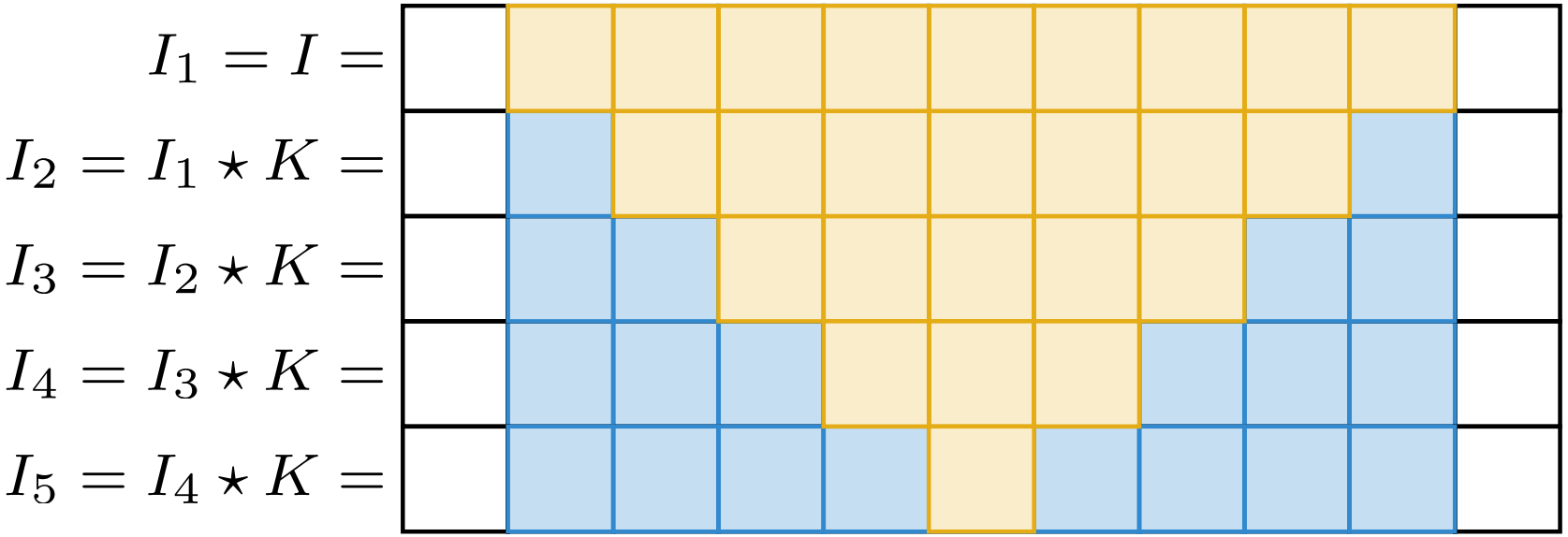
$I_4 = I_3 \star K =$

$I_5 = I_4 \star K =$

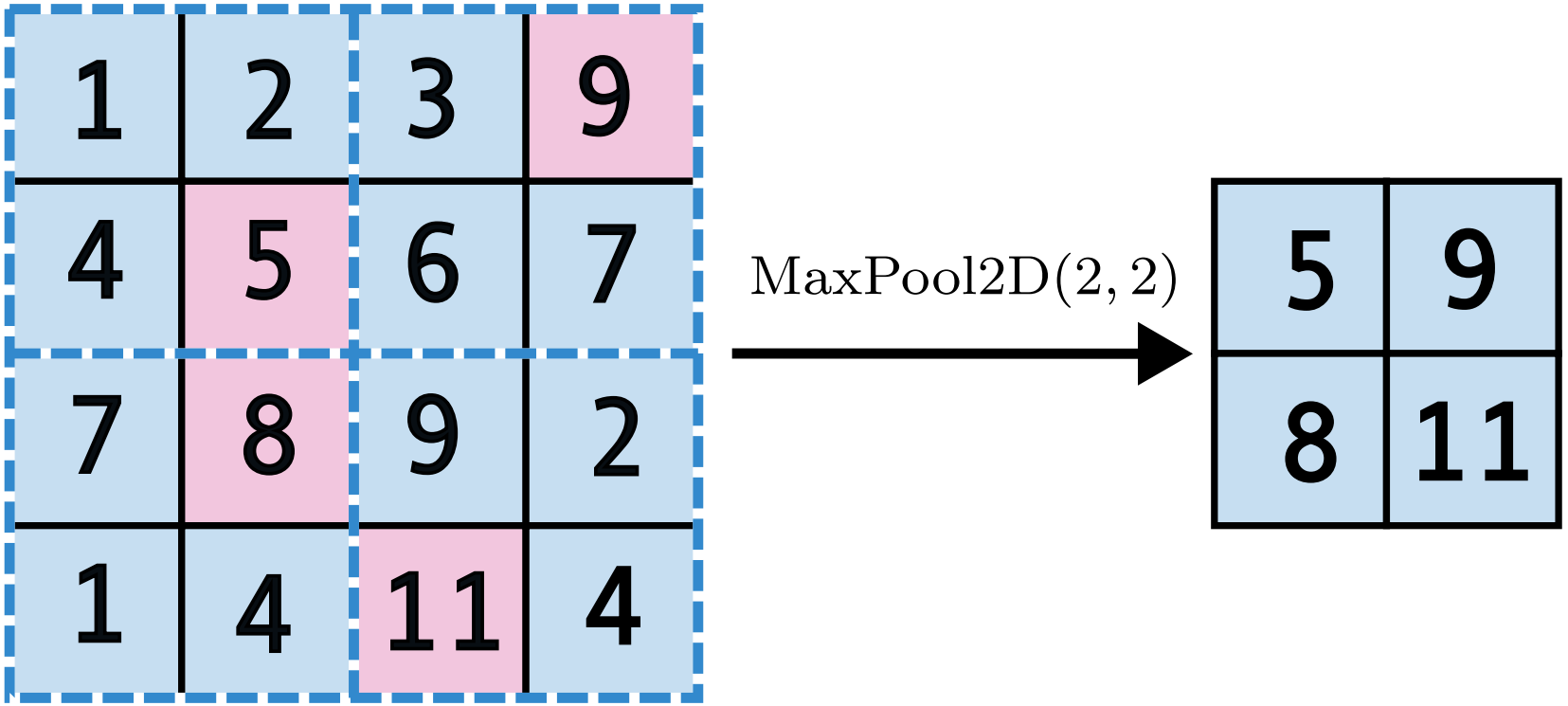


Receptive field

$K =$



Downsampling: MaxPool



[8] — *MaxPool2d — PyTorch 2.6 documentation*

Downsampling: outras estratégias

- Average Pooling
- Min Pooling
- Adaptive (Min/Max/Avg) Pooling
- Usar **stride** > 1

Normalização: BatchNorm

- Calculamos uma estimativa da média e variância de cada *minibatch*

$$\mu_B = \frac{1}{|\mathcal{B}|} \sum_{x \in \mathcal{B}} x$$

$$\sigma_B^2 = \frac{1}{|\mathcal{B}|} \sum_{x \in \mathcal{B}} (x - \mu_B)^2$$

- Para cada elemento x_i do lote $\mathcal{B}$, aplicamos a normalização:

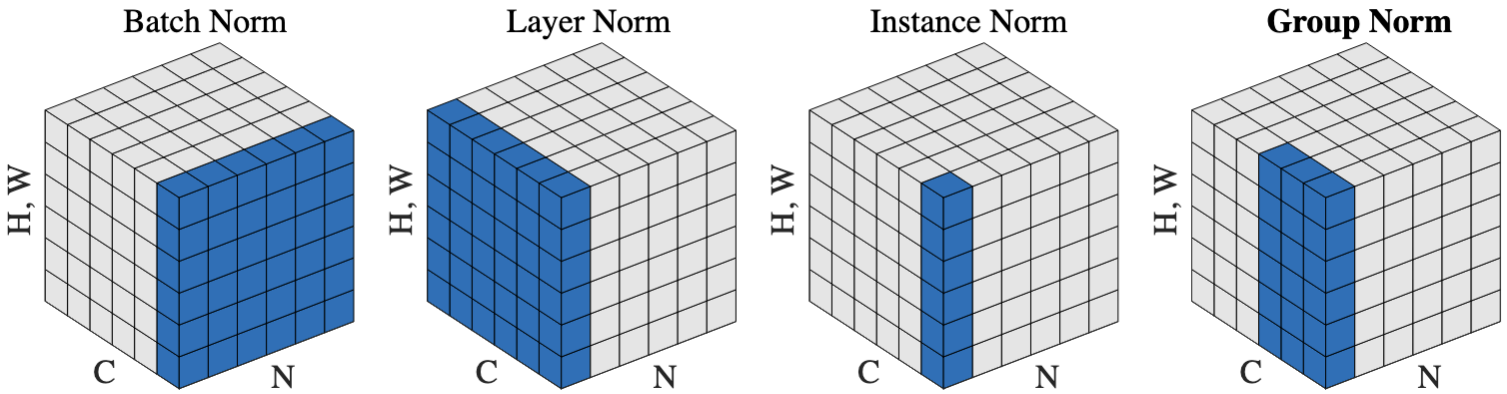
$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

$$y_i = \gamma \odot \hat{x}_i + \beta$$

- Onde γ e β são parâmetros que permitem que a rede reescale e desloque os dados normalizados.

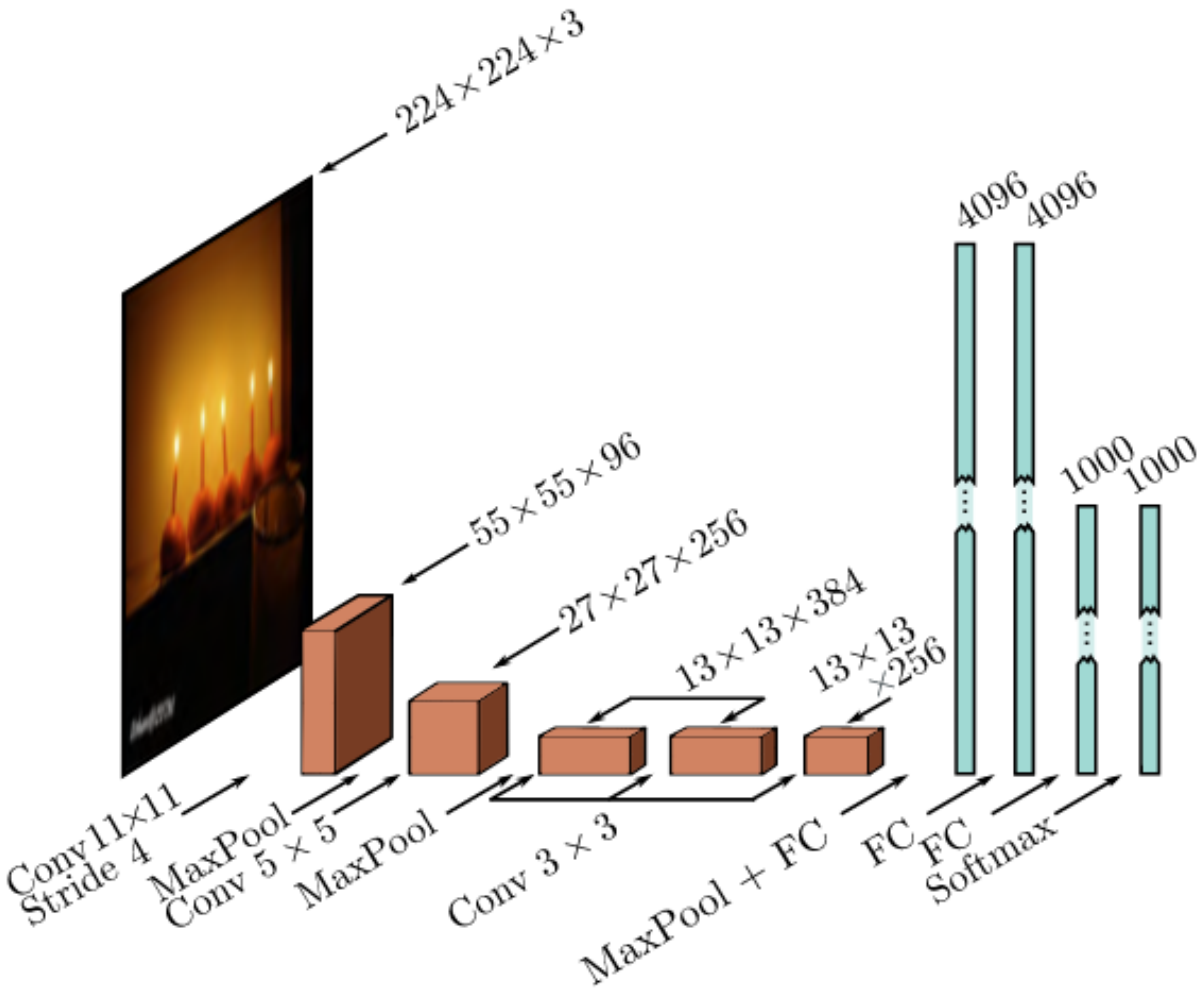
[6] 2015 — Sergey Ioffe et al. — *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*
[1] — *BatchNorm2d — PyTorch 2.6 documentation*

Outras Normalizações



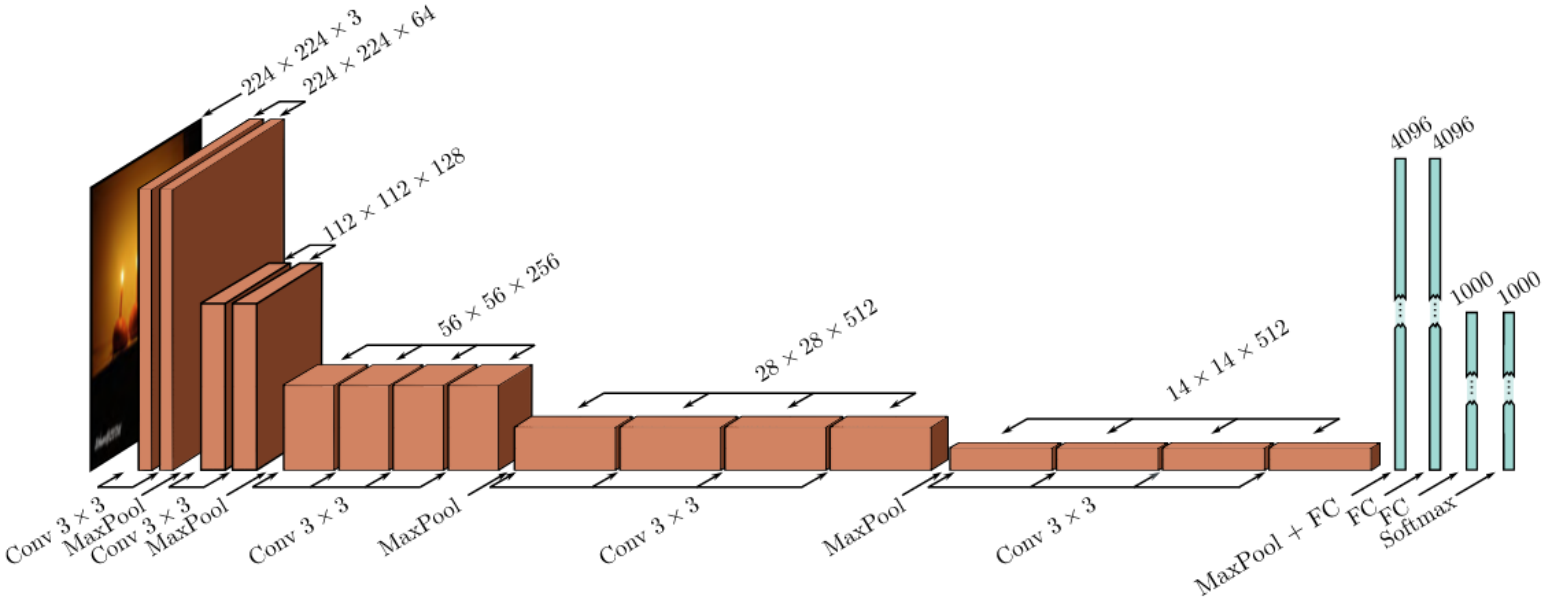
[16] 2018 — Yuxin Wu et al. — *Group Normalization*

AlexNet



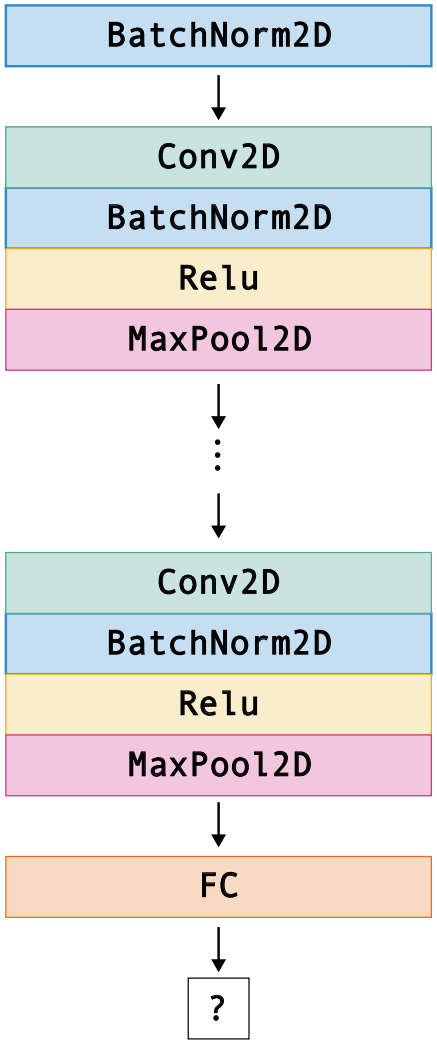
[7] 2012 — Alex Krizhevsky et al. — *ImageNet Classification with Deep Convolutional Neural Networks*
[10] 2023 — Simon Prince — *Understanding Deep Learning*

VGG



[13] 2015 — Karen Simonyan et al. — *Very Deep Convolutional Networks for Large-Scale Image Recognition*
[10] 2023 — Simon Prince — *Understanding Deep Learning*

Rede convolucional de classificação



- No caso de classificação binária, podemos terminar a rede com uma camada densa com 1 neurônio e ativação *sigmoid*[11].
- Quando temos multiplas classes, podemos usar uma camada densa com K neurônios e ativação *softmax*[14]. Onde K é o número de classes.

[11] — *Sigmoid — PyTorch 2.6 documentation*
[14] — *Softmax — PyTorch 2.6 documentation*

Classificação Binária I

- $\mathcal{D} = \{(x_i, y_i); x_i \in \mathcal{X}, y_i \in \{0, 1\}\}_{i=1,\cdots,N}$ um conjunto de pares (imagem, rótulo) de treinamento.
- Se $f(x, \theta)$ é a saída da rede, podemos usar a função de ativação *sigmoid* para obter uma probabilidade:

$$P(y_i = 1|x_i, \theta) = \sigma(f(x_i, \theta)) = \frac{1}{1 + e^{-f(x_i,\theta)}}$$
$$P(y_i = 0|x_i, \theta) = 1 - P(y_i = 1|x_i, \theta)$$

- Desta forma a probabilidade de que $f(x_i, \theta)$ pertença a classe correta é dada por:

$$\begin{aligned} P(y_i|x_i, \theta) &= P(y_i = 1|x_i, \theta)^{y_i} P(y_i = 0|x_i, \theta)^{1-y_i} \\ &= \sigma(f(x_i, \theta))^{y_i} (1 - \sigma(f(x_i, \theta)))^{1-y_i} \end{aligned}$$

[2] — *BCELoss — PyTorch 2.6 documentation*

Classificação Binária II

- O estimador de máxima verossimilhança (MLE) é dado por:

$$
\begin{aligned}
\hat{\theta} &= \arg\max_{\theta} \prod_{i=1}^{N} P(y_i|x_i, \theta) \\
&= \arg\max_{\theta} \prod_{i=1}^{N} \sigma(f(x_i, \theta))^{y_i} (1 - \sigma(f(x_i, \theta)))^{1-y_i} \\
&= \arg\min_{\theta} - \sum_{i=1}^{N} y_i \log(\sigma(f(x_i, \theta))) + (1 - y_i) \log(1 - \sigma(f(x_i, \theta))) \\
&= \arg\min_{\theta} \sum_{i=1}^{N} \mathrm{BCELoss}(y_i, f(x_i, \theta))
\end{aligned}
$$

Classificação Multiclasses I

- Caso y_i seja um vetor (one-hot), podemos usar a função de ativação *softmax* para obter um vetor de probabilidades apartir do modelo

$$P(y_i|x_i,\theta) = \text{softmax}(f(x_i,\theta)) = \frac{e^{f(x_i,\theta)}}{\sum_{j=1}^{K} e^{f_j(x_i,\theta)}}$$
$$= [P(y_i = 1|x_i,\theta), P(y_i = 2|x_i,\theta), \cdots, P(y_i = K|x_i,\theta)]$$

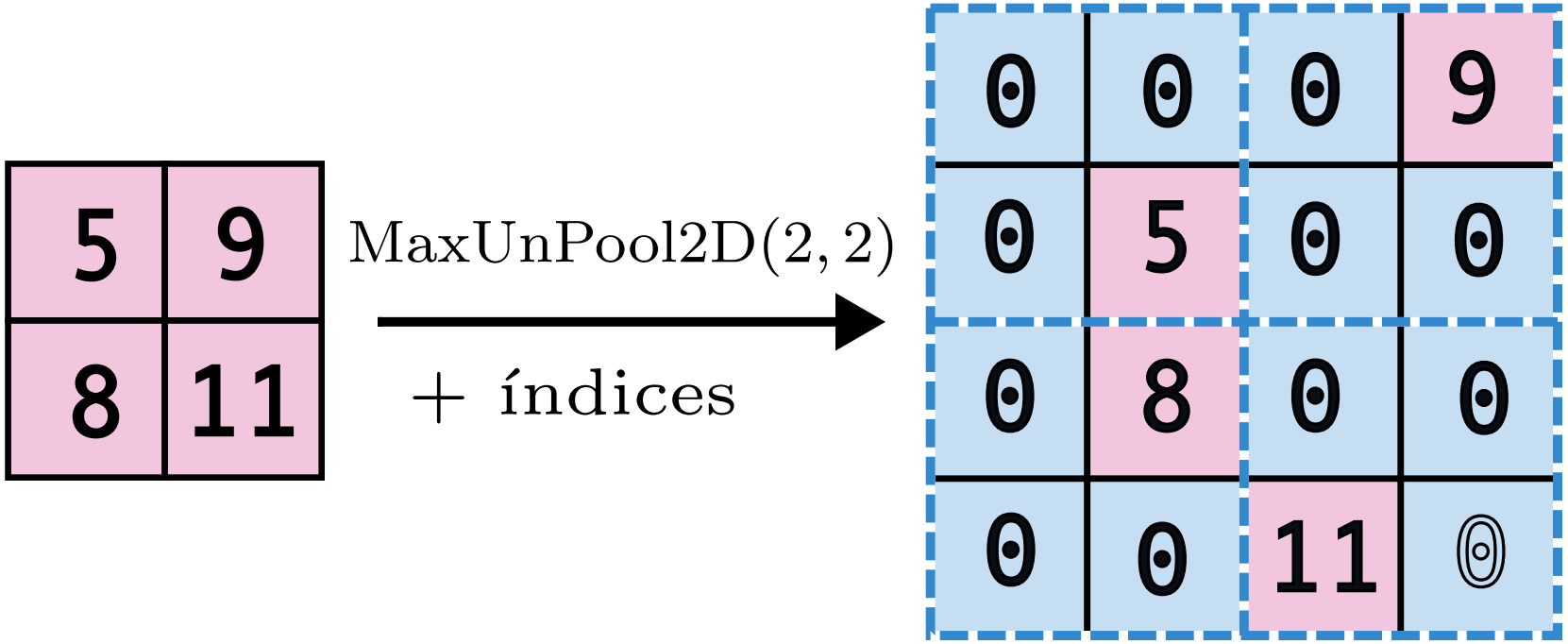
[5] — *CrossEntropyLoss — PyTorch 2.6 documentation*

Classificação Multiclasses II

- O estimador de máxima verossimilhança (MLE) é dado por:

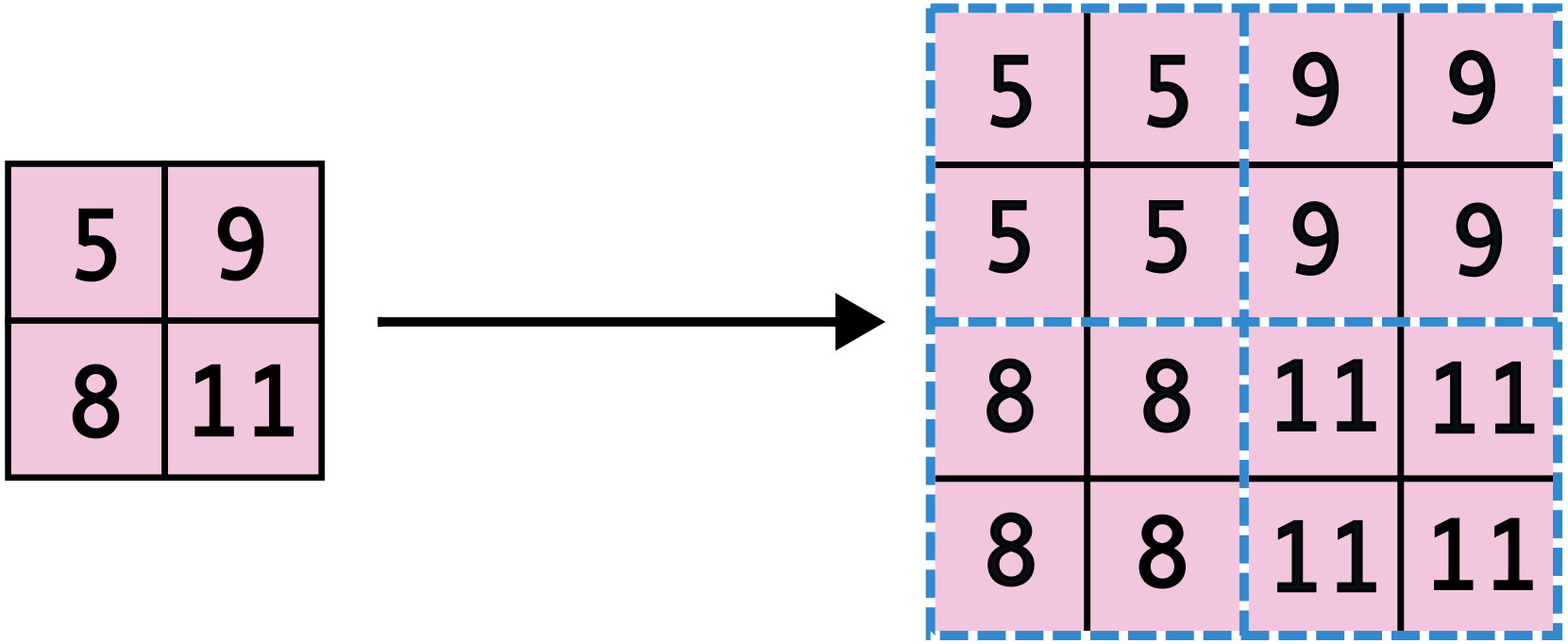
$$
\begin{aligned}
\hat{\theta} &= \arg\max_{\theta} \prod_{i=1}^{N} P(y_i|x_i,\theta) \\
&= \arg\max_{\theta} \prod_{i=1}^{N} \prod_{j=1}^{K} P(y_i = j|x_i,\theta)^{y_{ij}} \\
&= \arg\min_{\theta} -\sum_{i=1}^{N} \sum_{j=1}^{K} y_{ij} \log(P(y_i = j|x_i,\theta)) \\
&= \arg\min_{\theta} -\sum_{i=1}^{N} \sum_{j=1}^{K} y_{ij} \log(\mathrm{softmax}(f_j(x_i,\theta))) \\
&= \arg\min_{\theta} \sum_{i=1}^{N} \mathrm{CrossEntropyLoss}(y_i, f(x_i,\theta))
\end{aligned}
$$

Upsampling: Max Unpooling

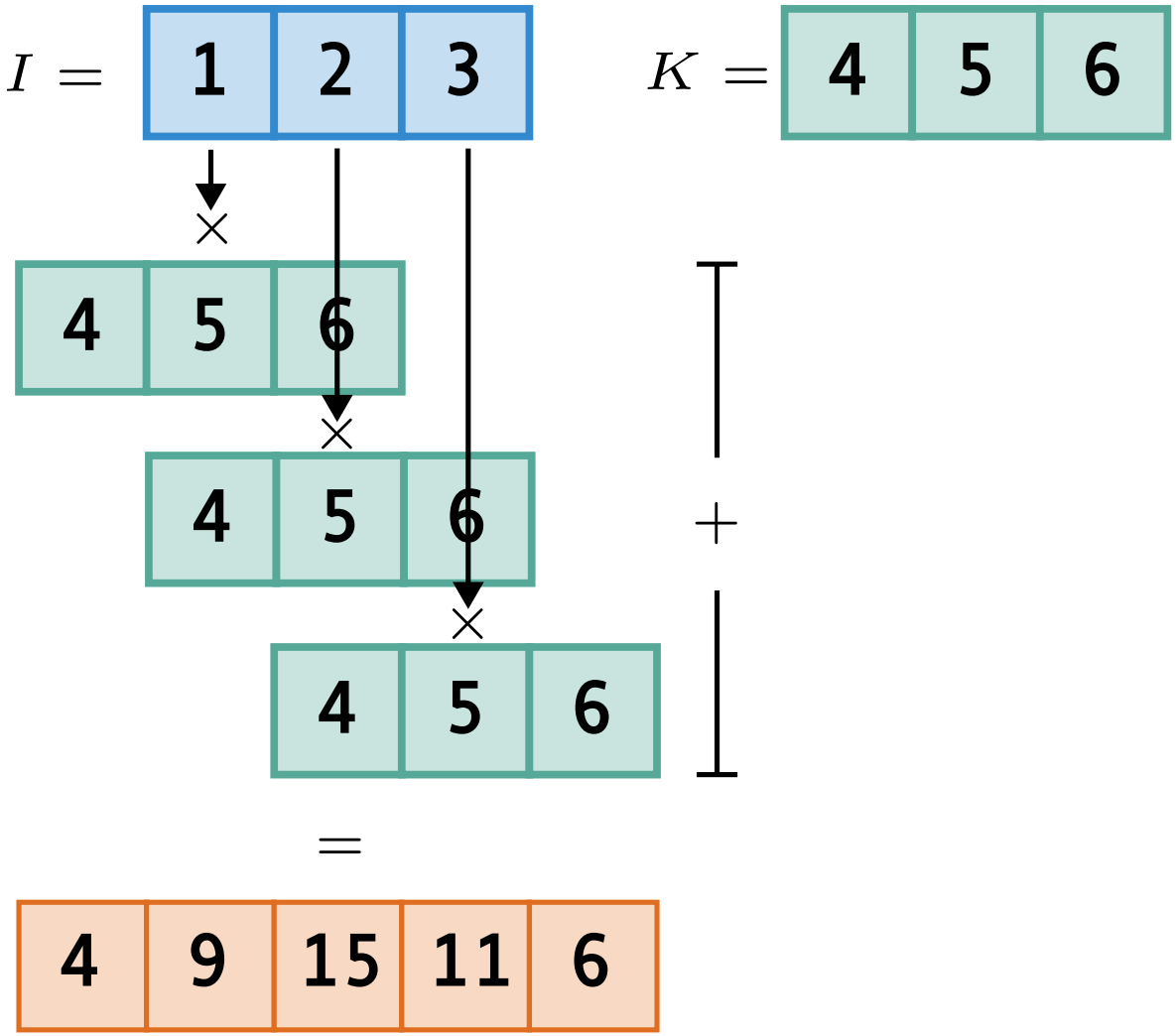


[9] — *MaxUnpool2d — PyTorch 2.6 documentation*

Upsampling: Dobrar a resolução



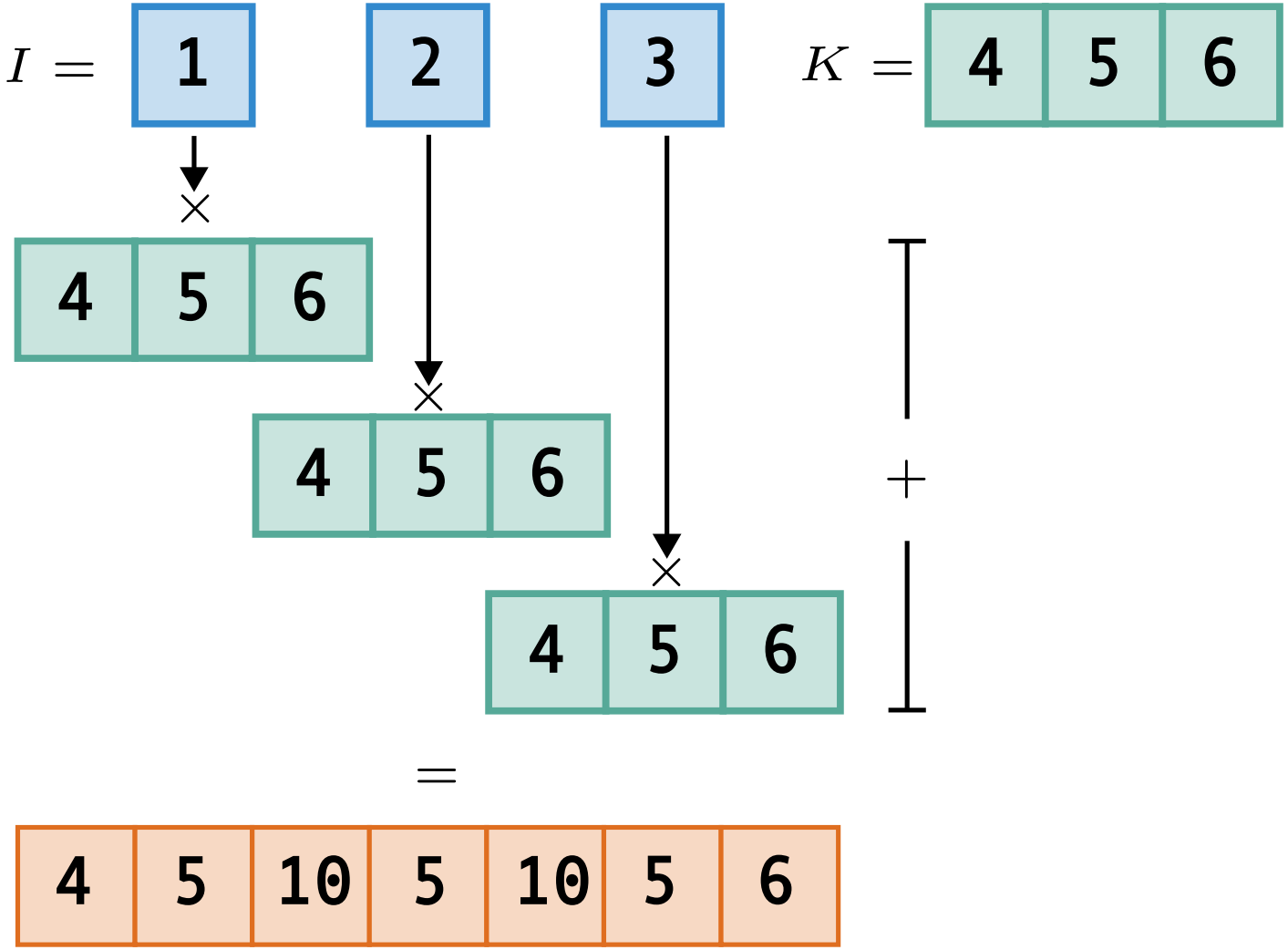
Upsampling: Convolução Transposta



[4] — *ConvTranspose2d — PyTorch 2.6 documentation*

PyTorch: [4]

Upsampling: Convolução Transposta + Stride



[4] — *ConvTranspose2d — PyTorch 2.6 documentation*

Convolução Transposta: Forma Matricial

- $I = [I_0, I_1, I_2]$
- $K = [K_0, K_1, K_2]$

$$\begin{bmatrix} O_0 \\ O_1 \\ O_2 \\ O_3 \\ O_4 \end{bmatrix} = \begin{bmatrix} K_0 & 0 & 0 \\ K_1 & K_0 & 0 \\ K_2 & K_1 & K_0 \\ 0 & K_2 & K_1 \\ 0 & 0 & K_2 \end{bmatrix} \begin{bmatrix} I_0 \\ I_1 \\ I_2 \end{bmatrix}$$

Referências I

[1] *BatchNorm2d — PyTorch 2.6 documentation*. Read_Status: New Read_Status_Date: 2025-03-31T14:06:19.618Z. URL: https://pytorch.org/docs/stable/generated/torch.nn.BatchNorm2d.html (visited on 03/31/2025).

[2] *BCELoss — PyTorch 2.6 documentation*. Read_Status: New Read_Status_Date: 2025-03-31T17:27:31.178Z. URL: https://pytorch.org/docs/stable/generated/torch.nn.BCELoss.html (visited on 03/31/2025).

[3] *Conv2d — PyTorch 2.6 documentation*. Read_Status: New Read_Status_Date: 2025-03-31T13:37:10.566Z. URL: https://pytorch.org/docs/stable/generated/torch.nn.Conv2d.html (visited on 03/31/2025).

[4] *ConvTranspose2d — PyTorch 2.6 documentation*. Read_Status: New Read_Status_Date: 2025-03-31T14:05:45.654Z. URL: https://pytorch.org/docs/stable/generated/torch.nn.ConvTranspose2d.html (visited on 03/31/2025).

Referências II

[5] *CrossEntropyLoss — PyTorch 2.6 documentation*. Read_Status: New Read_Status_Date: 2025-03-31T17:47:36.827Z. URL: https://pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html (visited on 03/31/2025).

[6] Sergey Ioffe and Christian Szegedy. "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift". In: *32nd International Conference on Machine Learning, ICML 2015* 1 (Feb. 10, 2015). ISBN: 9781510810587, pp. 448–456. arXiv: 1502.03167. URL: http://arxiv.org/abs/1502.03167.

[7] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. "ImageNet Classification with Deep Convolutional Neural Networks". In: *Advances in Neural Information Processing Systems*. Vol. 25. Read_Status: New Read_Status_Date: 2025-03-29T20:36:09.665Z. Curran Associates, Inc., 2012. URL: https://proceedings.neurips.cc/paper_files/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html (visited on 03/29/2025).

Referências III

[8] *MaxPool2d — PyTorch 2.6 documentation*. Read_Status: New Read_Status_Date: 2025-03-31T14:04:14.236Z. URL: https://pytorch.org/docs/stable/generated/torch.nn.MaxPool2d.html (visited on 03/31/2025).

[9] *MaxUnpool2d — PyTorch 2.6 documentation*. Read_Status: New Read_Status_Date: 2025-03-31T14:05:24.304Z. URL: https://pytorch.org/docs/stable/generated/torch.nn.MaxUnpool2d.html#torch.nn.MaxUnpool2d (visited on 03/31/2025).

[10] Simon Prince. *Understanding Deep Learning*. 2023. URL: https://udlbook.github.io/udlbook/ (visited on 12/20/2023).

[11] *Sigmoid — PyTorch 2.6 documentation*. Read_Status: New Read_Status_Date: 2025-03-31T16:07:14.251Z. URL: https://pytorch.org/docs/stable/generated/torch.nn.Sigmoid.html#torch.nn.Sigmoid (visited on 03/31/2025).

Referências IV

[12] Karen Simonyan, Andrea Vedaldi, and Andrew Zisserman. *Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps*. Read_Status: New Read_Status_Date: 2025-04-05T02:24:34.965Z. Apr. 19, 2014. DOI: 10.48550/arXiv.1312.6034. arXiv: 1312.6034[cs]. URL: http://arxiv.org/abs/1312.6034 (visited on 04/05/2025).

[13] Karen Simonyan and Andrew Zisserman. *Very Deep Convolutional Networks for Large-Scale Image Recognition*. Apr. 10, 2015. arXiv: 1409.1556[cs]. URL: http://arxiv.org/abs/1409.1556 (visited on 10/18/2023).

[14] *Softmax — PyTorch 2.6 documentation*. Read_Status: New Read_Status_Date: 2025-03-31T16:07:37.587Z. URL: https://pytorch.org/docs/stable/generated/torch.nn.Softmax.html#torch.nn.Softmax (visited on 03/31/2025).

[15] vdumoulin. *vdumoulin/conv_arithmetic*. original-date: 2016-02-24T15:18:33Z. Mar. 28, 2026. URL: https://github.com/vdumoulin/conv_arithmetic (visited on 03/29/2026).

Referências V

[16] Yuxin Wu and Kaiming He. *Group Normalization*. Read_Status: New Read_Status_Date: 2024-08-19T12:43:07.174Z. June 11, 2018. DOI: 10.48550/arXiv.1803.08494. arXiv: 1803.08494[cs]. URL: http://arxiv.org/abs/1803.08494 (visited on 08/16/2024).

[17] Matthew D. Zeiler and Rob Fergus. *Visualizing and Understanding Convolutional Networks*. ISBN: 9783319105895. Nov. 12, 2013. DOI: 10.1007/978-3-319-10590-1_53. arXiv: 1311.2901. URL: http://arxiv.org/abs/1311.2901.